

Table of Contents

Brute Force	2
Dynamic Programming Extras	8
Graphs Extra	29
Minimum-Cost-Perfect-Matching	53
Chinese Postman Problem	74

Brute Force

Print Apple Division

```
#include <algorithm>
#include <climits>
#include <iostream>
#include <numeric>
#include <vector>

#define ll long long

using namespace std;

int main(int argc, char *argv[]) {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);

    int n;
    cin >> n;
    vector<ll> a(n);
    for (auto &x : a) {
        cin >> x;
    }

    int m = n / 2;
    vector<ll> L, R;

    for (int i = 0; i < (1 << m); i++) {
        ll s = 0;
        for (int j = 0; j < m; j++) {
            if (i >> j & 1) {
                s += a[j];
            }
        }
        L.push_back(s);
    }

    for (int i = 0; i < (1 << (n - m)); i++) {
        ll s = 0;
        for (int j = 0; j < n - m; j++) {
            if (i >> j & 1) {
                s += a[m + j];
            }
        }
        R.push_back(s);
    }
}
```

```

sort(R.begin(), R.end());
ll tot = accumulate(a.begin(), a.end(), 0LL);
ll best = LLONG_MAX;

for (ll x : L) {
    ll need = tot / 2 - x;
    auto it = lower_bound(R.begin(), R.end(), need);
    if (it != R.end()) {
        best = min(best, abs(tot - 2 * (x + *it)));
    }
    if (it != R.begin()) {
        best = min(best, abs(tot - 2 * (x + *prev(it))));
    }
}

cout << best << "\n";

return 0;
}

```

Print Apple Division - DP

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n))
        return 0;
    vector<ll> a(n);
    ll S = 0;
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
        S += a[i];
    }

    const ll DP_THRESHOLD =
        50'000'000LL; // sum threshold for DP (tunable; uses ~O(S) memory)

    vector<char> chosen(n, 0); // will mark items in group A

    if (S <= DP_THRESHOLD) {
        // Dynamic programming pseudo-polynomial
        int Sz = (int)(S + 1);
    }
}

```

```

vector<char> dp(Sz, 0);
vector<int> parent(Sz, -1); // parent[s] = index of item used to reach s; -1
                             // = unreachable; -2 = base 0

dp[0] = 1;
parent[0] = -2;
for (int i = 0; i < n; ++i) {
    ll w = a[i];
    for (ll s = S; s >= w; --s) {
        if (!dp[(size_t)s] && dp[(size_t)(s - w)]) {
            dp[(size_t)s] = 1;
            parent[(size_t)s] = i;
        }
    }
}

ll target = S / 2;
ll best = 0;
for (ll s = target; s >= 0; --s) {
    if (dp[(size_t)s]) {
        best = s;
        break;
    }
}

// reconstruct
ll cur = best;
while (cur > 0) {
    int idx = parent[(size_t)cur];
    if (idx < 0)
        break; // safety
    chosen[idx] = 1;
    cur -= a[idx];
}

// build groups and print
vector<ll> A, B;
ll sumA = 0, sumB = 0;
for (int i = 0; i < n; ++i) {
    if (chosen[i]) {
        A.push_back(a[i]);
        sumA += a[i];
    } else {
        B.push_back(a[i]);
        sumB += a[i];
    }
}

cout << llabs(sumA - sumB) << '\n';
// print groups (weights). Format: first line is group A items, second line
// group B items. (You can change output formatting if you prefer indices or
// counts.)
for (size_t i = 0; i < A.size(); ++i) {
    if (i)
        cout << ' ';
    cout << A[i];
}
cout << '\n';

```

```

for (size_t i = 0; i < B.size(); ++i) {
    if (i)
        cout << ' ';
    cout << B[i];
}
cout << '\n';
return 0;
}

```

Suffix and Bounding

```

// suffix linear

// lexicographic order for pairs
inline bool leq(int a1, int a2, int b1, int b2) {
    return (a1 < b1 || a1 == b1 && a2 <= b2);
}

// and triples
inline bool leq(int a1, int a2, int a3, int b1, int b2, int b3) {
    return (a1 < b1 || a1 == b1 && leq(a2, a3, b2, b3));
} // and triples

// stably sort a[0..n-1] to b[0..n-1] with keys in 0..K from r
static void radixPass(int *a, int *b, int *r, int n,
                     int K) { // count occurrences
    int *c = new int[K + 1]; // counter array
    for (int i = 0; i <= K; i++)
        c[i] = 0; // reset counters
    for (int i = 0; i < n; i++)
        c[r[a[i]]]++; // count occurrences
    for (int i = 0, sum = 0; i <= K; i++) // exclusive prefix sums
    {
        int t = c[i];
        c[i] = sum;
        sum += t;
    }
    for (int i = 0; i < n; i++)
        b[c[r[a[i]]]++] = a[i]; // sort
    delete[] c;
}

// find the suffix array SA of s[0..n-1] in {1..K}^n
// require s[n]=s[n+1]=s[n+2]=0, n>=2
void suffixArray(int *s, int *SA, int n, int K) {
    int n0 = (n + 2) / 3, n1 = (n + 1) / 3, n2 = n / 3, n02 = n0 + n2;
    int *s12 = new int[n02 + 3];
}

```

```

s12[n02] = s12[n02 + 1] = s12[n02 + 2] = 0;
int *SA12 = new int[n02 + 3];
SA12[n02] = SA12[n02 + 1] = SA12[n02 + 2] = 0;
int *s0 = new int[n0];
int *SA0 = new int[n0];
// generate positions of mod 1 and mod 2 suffixes
// the "(n0-n1)" adds a dummy mod 1 suffix if n%3 == 1
for (int i = 0, j = 0; i < n + (n0 - n1); i++)
    if (i % 3 != 0)
        s12[j++] = i;
// lsb radix sort the mod 1 and mod 2 triples
radixPass(s12, SA12, s + 2, n02, K);
radixPass(SA12, s12, s + 1, n02, K);
radixPass(s12, SA12, s, n02, K);
// find lexicographic names of triples
int name = 0, c0 = -1, c1 = -1, c2 = -1;
for (int i = 0; i < n02; i++) {
    if (s[SA12[i]] != c0 || s[SA12[i] + 1] != c1 || s[SA12[i] + 2] != c2) {
        name++;
        c0 = s[SA12[i]];
        c1 = s[SA12[i] + 1];
        c2 = s[SA12[i] + 2];
    }
    if (SA12[i] % 3 == 1)
        s12[SA12[i] / 3] = name; // left half
    else
        s12[SA12[i] / 3 + n0] = name; // right half
}
// recurse if names are not yet unique
if (name < n02) {
    suffixArray(s12, SA12, n02, name);
    // store unique names in s12 using the suffix array
    for (int i = 0; i < n02; i++)
        s12[SA12[i]] = i + 1;
} else // generate the suffix array of s12 directly
    for (int i = 0; i < n02; i++)
        SA12[s12[i] - 1] = i;
// stably sort the mod 0 suffixes from SA12 by their first character
for (int i = 0, j = 0; i < n02; i++)
    if (SA12[i] < n0)
        s0[j++] = 3 * SA12[i];
radixPass(s0, SA0, s, n0, K);
// merge sorted SA0 suffixes and sorted SA12 suffixes
for (int p = 0, t = n0 - n1, k = 0; k < n; k++) {
#define GetI() (SA12[t] < n0 ? SA12[t] * 3 + 1 : (SA12[t] - n0) * 3 + 2)
    int i = GetI(); // pos of current offset 12 suffix
    int j = SA0[p]; // pos of current offset 0 suffix
    if (SA12[t] < n0
        ? // different compares for mod 1 and mod 2 suffixes
        leq(s[i], s12[SA12[t] + n0], s[j], s12[j / 3])
        : leq(s[i], s[i + 1], s12[SA12[t] - n0 + 1], s[j], s[j + 1],
            s12[j / 3 + n0])) { // suffix from SA12 is smaller
        SA[k] = i;
        t++;
    }
}

```

```

        if (t == n02) // done --- only SA0 suffixes left
            for (k++; p < n0; p++, k++)
                SA[k] = SA0[p];
    } else { // suffix from SA0 is smaller
        SA[k] = j;
        p++;
        if (p == n0) // done --- only SA12 suffixes left
            for (k++; t < n02; t++, k++)
                SA[k] = GetI();
    }
}
delete[] s12;
delete[] SA12;
delete[] SA0;
delete[] s0;
}

int main() {

    // MIGHT NOT WORK FOR EMPTY STRINGS AND 1 CHARACTER STRINGS
    return 0;
}

// ----

// bounding box code.

// Function to calculate a dynamic bounding box based on input points.
Point calculate_bounding_box(const std::vector<Point> &points) {
    long double maxX = -inf, maxY = -inf;
    long double minX = inf, minY = inf;

    // Traverse all points to find the bounding box
    for (const Point &pt : points) {
        if (pt.x > maxX)
            maxX = pt.x;
        if (pt.x < minX)
            minX = pt.x;
        if (pt.y > maxY)
            maxY = pt.y;
        if (pt.y < minY)
            minY = pt.y;
    }

    // Use the most extreme points to define the bounding box
    Point bbox((maxX - minX) * 2,
               (maxY - minY) * 2); // Making the box larger for stability
    return bbox;
}

```

Dynamic Programming

All LCS

```
#include <bits/stdc++.h>
using namespace std;

// Build LCS length table C for strings a (length m) and b (length n).
vector<vector<int>> buildLCSMatrix(const string &a, const string &b) {
    int m = (int)a.size();
    int n = (int)b.size();
    vector<vector<int>> C(m + 1, vector<int>(n + 1, 0));
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (a[i - 1] == b[j - 1])
                C[i][j] = C[i - 1][j - 1] + 1;
            else
                C[i][j] = max(C[i - 1][j], C[i][j - 1]);
        }
    }
    return C;
}

// Helper to create a unique key for memo map from (i,j).
inline long long keyFor(int i, int j) {
    return ((long long)i << 32) | (unsigned long long)j;
}

// backtrackAll returns a set of all distinct LCS strings for prefixes
// a[0..i-1], b[0..j-1]. C is the DP matrix (m+1 x n+1). Uses memoization to
// avoid repeated work.
set<string> backtrackAll(const vector<vector<int>> &C, const string &a,
                        const string &b, int i, int j,
                        unordered_map<long long, set<string>> &memo) {
    long long key = keyFor(i, j);
    auto it = memo.find(key);
    if (it != memo.end())
        return it->second;

    set<string> result;
    if (i == 0 || j == 0) {
        result.insert(string("")); // base: empty string
    } else if (a[i - 1] == b[j - 1]) {
        // characters match: append that char to all strings returned from (i-1,j-1)
        set<string> prev = backtrackAll(C, a, b, i - 1, j - 1, memo);
        for (const string &s : prev)
            result.insert(s + a[i - 1]);
    } else {
```

```

// follow both directions that are tied for optimal
if (C[i][j - 1] >= C[i - 1][j]) {
    set<string> s1 = backtrackAll(C, a, b, i, j - 1, memo);
    result.insert(s1.begin(), s1.end());
}
if (C[i - 1][j] >= C[i][j - 1]) {
    set<string> s2 = backtrackAll(C, a, b, i - 1, j, memo);
    result.insert(s2.begin(), s2.end());
}
}

memo.emplace(key, result);
return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string A, B;
    // Read two lines from stdin as the two strings.
    // (Use getline to allow spaces in the strings.)
    cout << "Enter first string (A):\n";
    if (!getline(cin, A))
        return 0;
    cout << "Enter second string (B):\n";
    if (!getline(cin, B))
        return 0;

    auto C = buildLCSMatrix(A, B);
    int m = (int)A.size();
    int n = (int)B.size();
    int lcs_len = C[m][n];

    cout << "\nLCS length = " << lcs_len << "\n";

    // If length is 0, one LCS is the empty string.
    if (lcs_len == 0) {
        cout << "Only LCS is the empty string.\n";
        return 0;
    }

    unordered_map<long long, set<string>> memo;
    set<string> all = backtrackAll(C, A, B, m, n, memo);

    // Filter out the empty string (shouldn't happen for positive length), and
    // only keep strings of length lcs_len.
    vector<string> results;
    results.reserve(all.size());
    for (const auto &s : all) {
        if ((int)s.size() == lcs_len)
            results.push_back(s);
    }
}

```

```

sort(results.begin(), results.end()); // ensure deterministic order
cout << "Number of distinct LCSs: " << results.size() << "\n\n";
cout << "All LCS strings (each on its own line):\n";
for (const auto &s : results)
    cout << s << "\n";

return 0;
}

```

Count Distinct Subsequences

// Given a string str of length n, your task is to find the count of distinct
// subsequences of it.

// Examples:

// Input: str = "gfg"

// Output: 7

// Explanation: The seven distinct subsequences are "", "g", "f", "gf", "fg",
// "gg" and "gfg"

// Input: str = "ggg"

// Output: 4

// Explanation: The four distinct subsequences are "", "g", "gg" and "ggg"

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int distinctSubseq(string &str) {
```

```
    int n = str.size();
```

```
    // to store the last occurrence
```

```
    // of each character in the string
```

```
    vector<int> last(26, 0);
```

```
    // to store result after each index
```

```
    int res = 1;
```

```
    for (int i = 1; i <= n; i++) {
```

```
        // double the count of unique subsequences
```

```
        // and remove the repetition
```

```
        int cur = 2 * res - last[str[i - 1] - 'a'];
```

```
        // update the last occurrence of the character
```

```
        last[str[i - 1] - 'a'] = res;
```

```
        res = cur;
```

```
    }  
    return res;  
}
```

LCS - Count

```
#include <bits/stdc++.h>  
using namespace std;  
  
int lcs(string &s1, string &s2) {  
  
    int m = s1.length(), n = s2.length();  
  
    // dp vector is initialized to all zeros  
    // This vector stores the LCS values for the current row.  
    // dp[j] represents LCS of s1[0..i] and s2[0..j]  
    vector<int> dp(n + 1, 0);  
  
    // i and j represent the lengths of s1 and s2 respectively  
    for (int i = 1; i <= m; ++i) {  
  
        // prev stores the value from the previous  
        // row and previous column (i-1), (j -1)  
        // Used to keep track of LCS[i-1][j-1] while updating dp[j]  
        int prev = dp[0];  
  
        for (int j = 1; j <= n; ++j) {  
  
            // temp temporarily stores the current  
            // dp[j] before it gets updated  
            int temp = dp[j];  
  
            // If characters match, add 1 to the value  
            // from the previous row and previous column  
            // dp[j] = 1 + LCS[i-1][j-1]  
            if (s1[i - 1] == s2[j - 1])  
                dp[j] = 1 + prev;  
            else  
                // Otherwise, take the maximum of the  
                // left (dp[j-1]) and top (dp[j]) values  
                dp[j] = max(dp[j - 1], dp[j]);  
  
            // Update prev for the next iteration  
            // This keeps the value of the previous  
            // row (i-1) for future comparisons  
            prev = temp;  
        }  
    }  
}
```

```

    }

    // The last element of the vector contains the length of the LCS
    // dp[n] stores the length of LCS of s1[0..m] and s2[0..n]
    return dp[n];
}

```

LCS - Count of 3

```

#include <bits/stdc++.h>
using namespace std;

// Function to compute LCS of three strings using 2D DP (space optimized)
int lcsOf3(string &s1, string &s2, string &s3) {

    // Length of first string
    int n1 = s1.length();
    // Length of second string
    int n2 = s2.length();
    // Length of third string
    int n3 = s3.length();

    // Initialize two 2D arrays for DP:
    // prev holds values for previous i-1 level
    // curr holds values for current i level
    vector<vector<int>> prev(n2 + 1, vector<int>(n3 + 1, 0));
    vector<vector<int>> curr(n2 + 1, vector<int>(n3 + 1, 0));

    // Iterate over all characters of s1
    for (int i = 1; i <= n1; i++) {

        // Iterate over all characters of s2
        for (int j = 1; j <= n2; j++) {

            // Iterate over all characters of s3
            for (int k = 1; k <= n3; k++) {

                // If current characters of all three strings match
                if (s1[i - 1] == s2[j - 1] && s2[j - 1] == s3[k - 1])
                    curr[j][k] = 1 + prev[j - 1][k - 1];
                else
                    // Take the maximum of excluding current
                    // character from any one string
                    curr[j][k] = max({prev[j][k], curr[j - 1][k], curr[j][k - 1]});
            }
        }
    }

    // Move curr to prev for the next iteration
}

```

```
    prev = curr;
}

// The result is in curr[n2][n3], which holds the final LCS length
return curr[n2][n3];
}
```

Longest Common Substring - Count

```
#include <iostream>
#include <vector>
using namespace std;

// Given two strings 's1' and 's2', find the length of the longest common
// substring.
//
// Example:
//
// Input: s1 = "GeeksforGeeks", s2 = "GeeksQuiz"
// Output : 5
// Explanation:
// The longest common substring is "Geeks" and is of length 5.

int longestCommonSubstr(const string &s1, const string &s2) {
    int m = s1.length();
    int n = s2.length();

    // Create a 1D array to store the previous row's results
    vector<int> prev(n + 1, 0);

    int res = 0;
    for (int i = 1; i <= m; i++) {

        // Create a temporary array to store the current row
        vector<int> curr(n + 1, 0);
        for (int j = 1; j <= n; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                curr[j] = prev[j - 1] + 1;
                res = max(res, curr[j]);
            } else {
                curr[j] = 0;
            }
        }
    }

    // Move the current row's data to the previous row
    prev = curr;
}
```

```
    return res;
}
```

Longest Common Supersequence - Count

```
// Given two strings s1 and s2, the task is to
// find the length of the shortest string
// that has both s1 and s2 as subsequences.

// Input: s1 = "geek", s2 = "eke"
// Output: 5
// Explanation: String "geeke" has both string "geek" and "eke" as subsequences.

#include <bits/stdc++.h>
using namespace std;

// Returns length of LCS for s1[0..m-1], s2[0..n-1]
int lcs(string &s1, string &s2) {
    int m = s1.size();
    int n = s2.size();

    // Initializing a matrix of size (m+1)*(n+1)
    vector<vector<int>>> dp(m + 1, vector<int>(n + 1, 0));

    // Building dp[m+1][n+1] in bottom-up fashion
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (s1[i - 1] == s2[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    // dp[m][n] contains length of LCS for s1[0..m-1]
    // and s2[0..n-1]
    return dp[m][n];
}

int shortestCommonSupersequence(string &s1, string &s2) {

    return s1.size() + s2.size() - lcs(s1, s2);
}
```

Longest Palindromic Subsequence

```
#include <bits/stdc++.h>

// Given a string s, find the length of the Longest Palindromic Subsequence in
// it.

using namespace std;

// Function to find the length of the lps
int longestPalinSubseq(const string &s) {
    int n = s.size();

    // Create two vectors: one for the current state (dp)
    // and one for the previous state (dpPrev)
    vector<int> curr(n), prev(n);

    // Loop through the string in reverse (starting from the end)
    for (int i = n - 1; i >= 0; --i) {

        // Initialize the current state of dp
        curr[i] = 1;

        // Loop through the characters ahead of i
        for (int j = i + 1; j < n; ++j) {

            // If the characters at i and j are the same
            if (s[i] == s[j]) {

                // Add 2 to the length of the palindrome between them
                curr[j] = prev[j - 1] + 2;
            } else {

                // Take the maximum between excluding either i or j
                curr[j] = max(prev[j], curr[j - 1]);
            }
        }

        // Update previous to the current state of dp
        prev = curr;
    }

    return curr[n - 1];
}

int main() {
    string s = "bbabcbcab";
    cout << longestPalinSubseq(s);
    return 0;
}
```

Longest Palindromic Substring

// Given a string s, find the longest substring which is a palindrome. If there are multiple answers, then find the first appearing substring.

// Examples:

// Input: s = "forgeeksskeegfor"

// Output: "geeksskeeg"

// Explanation: The longest substring that reads the same forward and backward is "geeksskeeg". Other palindromes like "kssk" or "eeksskee" are shorter.

// Input: s = "Geeks"

// Output: "ee"

// Explanation: The substring "ee" is the longest palindromic part in "Geeks".

// All others are shorter single characters.

// Input: s = "abc"

// Output: "a"

// Explanation: No multi-letter palindromes exist. So the first character "a" is returned as the longest palindromic substring.

```
#include <iostream>
```

```
#include <string>
```

```
#include <vector>
```

```
using namespace std;
```

// Transforms the input string with sentinels and separators

```
string transformString(const string &s) {
```

```
    string ms = "@"; // left sentinel to avoid bounds check
```

```
    for (char c : s) {
```

```
        ms += "#" + string(1, c); // insert '#' between every character
```

```
    }
```

```
    ms += "#$"; // right sentinel
```

```
    return ms;
```

```
}
```

// Runs Manacher's algorithm and returns the p array

```
vector<int> runManacher(const string &ms) {
```

```
    int n = ms.size();
```

```
    vector<int> p(n, 0);
```

```
    int l = 0, r = 0;
```

```
    for (int i = 1; i < n - 1; ++i) {
```

```
        // initialize p[i] based on its mirror and current [l, r] range
```

```
        if (r + 1 - i >= 0 && r + 1 - i < n)
```

```
            p[i] = max(0, min(r - i, p[r + 1 - i]));
```

```
        // try expanding around center i
```

```
        while (ms[i + 1 + p[i]] == ms[i - 1 - p[i]]) {
```

```

        ++p[i];
    }

    // update [l, r] if the new palindrome goes beyond current right boundary
    if (i + p[i] > r) {
        l = i - p[i];
        r = i + p[i];
    }
}
return p;
}

// Returns the radius of the longest palindrome centered at original index 'cen'
int getLongest(const vector<int> &p, int cen, int odd) {
    int pos = 2 * cen + 2 + !odd;
    return p[pos];
}

// Checks whether the substring s[l..r] is a palindrome
bool checkPalindrome(const vector<int> &p, int l, int r) {
    int res = getLongest(p, (r + l) / 2, (r - l + 1) % 2);
    return (r - l + 1) <= res;
}

// Finds and returns the longest palindromic substring in s
string getLongestPal(const string &s) {
    if (s.empty())
        return "";

    int n = s.size(), maxLen = 1, start = 0;

    // Transform the string and run Manacher's algorithm
    string ms = transformString(s);
    vector<int> p = runManacher(ms);

    for (int i = 0; i < n; ++i) {
        int oddLen = getLongest(p, i, 1);
        if (oddLen > maxLen) {
            // update start for odd-length palindrome
            start = i - (oddLen - 1) / 2;
            maxLen = oddLen;
        }

        int evenLen = getLongest(p, i, 0);
        if (evenLen > maxLen) {
            // update start for even-length palindrome
            start = i - (evenLen - 1) / 2;
            maxLen = evenLen;
        }
    }

    return s.substr(start, maxLen);
}

```

```
int main() {
    string s = "forgeeksskeegfor";
    cout << getLongestPal(s) << endl;
    return 0;
}
```

Longest Repeated Subsequence

```
// C++ program to find the longest repeating
// subsequence

// Given a string s, the task is to find the longest repeating subsequence, such
// that the two subsequences don't have the same string character at the same
// position, i.e. any ith character in the two subsequences shouldn't have the
// same index in the original string.
//
// Examples:
//
// Input: s = "aabb"
// Output: "ab"
// Explanation: The longest repeated subsequence is "ab", formed by matching 'a'
// from positions 1 and 2, and 'b' from positions 3 and 4.
//
// Input: s = "aab"
// Output: "a"
// Explanation: The two subsequences are 'a'(first) and 'a' (second). Note that
// 'b' cannot be considered as part of a subsequence as it would be at the same
// index in both.
//
// This problem is just the modification of the Longest Common Subsequence
// problem. The idea is to find the LCS(s, s) where s is the input string with
// the restriction that when both the characters are the same, they shouldn't be
// on the same index in the two strings.

#include <algorithm>
#include <iostream>
#include <string>
using namespace std;

int longestRepeatingSubsequence(string s) {
    int n = s.length();

    // Create and initialize DP table
    int dp[n + 1][n + 1];
    for (int i = 0; i <= n; i++)
        for (int j = 0; j <= n; j++)
            dp[i][j] = 0;
```

```

// Fill dp table (similar to LCS loops)
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {

        // If characters match and indexes are
        // not same
        if (s[i - 1] == s[j - 1] && i != j)
            dp[i][j] = 1 + dp[i - 1][j - 1];

        // If characters do not match
        else
            dp[i][j] = max(dp[i][j - 1], dp[i - 1][j]);
    }
}
return dp[n][n];
}

string longestRepeatedSubSeq(string s) {

    // THIS PART OF CODE IS
    // FILLS dp[][]
    int n = s.length();
    int dp[n + 1][n + 1];

    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= n; j++) {
            dp[i][j] = 0;
        }
    }

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (s[i - 1] == s[j - 1] && i != j) {
                dp[i][j] = 1 + dp[i - 1][j - 1];
            } else {
                dp[i][j] = max(dp[i][j - 1], dp[i - 1][j]);
            }
        }
    }

    // THIS PART OF CODE FINDS THE RESULT STRING USING DP[][]
    // Initialize result
    string res = "";

    // Traverse dp[][] from bottom right
    int i = n, j = n;
    while (i > 0 && j > 0) {

        // If this cell is same as diagonally
        // adjacent cell just above it, then
        // same characters are present at
        // str[i-1] and str[j-1]. Append any
        // of them to result.

```

```

    if (dp[i][j] == dp[i - 1][j - 1] + 1) {
        res = res + s[i - 1];
        i--;
        j--;
    }

    // Otherwise we move to the side
    // that gave us maximum result
    else if (dp[i][j] == dp[i - 1][j]) {
        i--;
    } else {
        j--;
    }
}

// Since we traverse dp[][] from bottom,
// we get result in reverse order.
reverse(res.begin(), res.end());

return res;
}

```

Minimum insertions to form a palindrome

```

// Given a string s, the task is to find the minimum number of characters to be
// inserted to convert it to a palindrome.

// Input: s = "geeks"
// Output: 3
// Explanation: "skgeegks" is a palindromic string, which requires 3 insertions.

#include <bits/stdc++.h>
using namespace std;

// Function to find minimum insertions
// to make string palindrome
int findMinInsertions(string &s) {
    int n = s.size();
    vector<int> dp(n, 0);

    // Iterate over each character from right to left
    for (int l = n - 2; l >= 0; l--) {

        // This represents dp[l+1][h-1] from the previous row
        int prev = 0;
        for (int h = l + 1; h < n; h++) {

```

```
// Save current dp[h] before overwriting
int temp = dp[h];

if (s[l] == s[h]) {

    // No new insertion needed if characters match
    dp[h] = prev;
} else {

    // Take min of two cases + 1
    dp[h] = min(dp[h], dp[h - 1]) + 1;
}

// Update prev for the next column
prev = temp;
}
}

return dp[n - 1];
}
```

Optimal Edit Distance

```
// C++ program to find minimum number
// of operations to convert s1 to s2
#include <bits/stdc++.h>
using namespace std;

int editDistance(string &s1, string &s2) {
    int m = s1.size();
    int n = s2.size();

    int prev;
    vector<int> curr(n + 1, 0);

    for (int j = 0; j <= n; j++)
        curr[j] = j;

    for (int i = 1; i <= m; i++) {
        prev = curr[0];
        curr[0] = i;
        for (int j = 1; j <= n; j++) {
            int temp = curr[j];
            if (s1[i - 1] == s2[j - 1])
                curr[j] = prev;
            else
                curr[j] = 1 + min({curr[j - 1], prev, curr[j]});
        }
    }
}
```

```
        prev = temp;
    }
}

return curr[n];
}
```

Optimal Edit Distance - Without Edit

```
#include <bits/stdc++.h>
using namespace std;

int lcs(string &s1, string &s2) {
    int m = s1.length(), n = s2.length();
    vector<vector<int>> dp(2, vector<int>(n + 1));

    for (int i = 0; i <= m; i++) {
        bool curr = i & 1;

        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0)
                dp[curr][j] = 0;

            else if (s1[i - 1] == s2[j - 1])
                dp[curr][j] = dp[1 - curr][j - 1] + 1;

            else
                dp[curr][j] = max(dp[1 - curr][j], dp[curr][j - 1]);
        }
    }

    return dp[m & 1][n];
}

int minOperations(string s1, string s2) {
    int m = s1.size();
    int n = s2.size();

    // the length of the LCS for s1[0..m-1] and s2[0..n-1]
    int len = lcs(s1, s2);

    // Characters to delete from s1
    int minDeletions = m - len;
}
```

```
// Characters to insert into s1
int minInsertions = n - len;

// Total operations needed
int total = minDeletions + minInsertions;
return total;
}
```

Optimal Knapsack

```
int KnapSack(int val[], int wt[], int n, int W) {
    // array to store final result
    // dp[i] stores the profit with KnapSack capacity "i"
    int dp[W + 1];

    // initially profit with 0 to W KnapSack capacity is 0
    memset(dp, 0, sizeof(dp));

    // iterate through all items
    for (int i = 0; i < n; i++)
        // traverse dp array from right to left
        for (int j = W; j >= wt[i]; j--)
            dp[j] = max(dp[j], val[i] + dp[j - wt[i]]);
    /*above line finds out maximum of dp[j](excluding ith
    element value) and val[i] + dp[j-wt[i]] (including ith
    element value and the
    profit with "KnapSack capacity - ith element weight")
    */
    return dp[W];
}
```

Perfect Sum

```
// Given an array arr[] of non-negative integers and an integer target. The task
// is to print all subsets of the array whose sum is equal to the given target.
// Note: If no subset has a sum equal to target, print -1.

// Examples:
```

```
// Input: arr[] = [5, 2, 3, 10, 6, 8], target = 10
// Output: [ [5, 2, 3], [2, 8], [10] ]
// Explanation: We need to find all subsets of arr[] that sum up to target = 10.
// Subset [5, 2, 3] - Sum = 5 + 2 + 3 = 10
// Subset [2, 8] - Sum = 2 + 8 = 10
// Subset [10] - Sum = 10

// Input: arr[] = [5, 7, 8], target = 3
// Output: [ [-1] ]
// Explanation: There are no subsets of the array that sum up to the target 3.

// Input: arr[] = [35, 2, 8, 22], target = 0
// Output: [ [ ] ]
// Explanation: The empty subset is the only subset with a sum of 0.
```

```
#include <bits/stdc++.h>
using namespace std;
```

```
// Recursively finds all subsets with the given target
void findSubsets(vector<int> &arr, vector<vector<bool>> &dp, int i, int target,
                vector<int> &curr, vector<vector<int>> &res) {
```

```
    // Base case: If target becomes 0
    if (i == 0) {
        if (target == 0)
            res.push_back(curr);
        if (arr[0] == target) {
            curr.push_back(arr[0]);
            res.push_back(curr);
            curr.pop_back();
        }
        return;
    }

    // Exclude current element
    if (dp[i - 1][target]) {
        findSubsets(arr, dp, i - 1, target, curr, res);
    }

    // Include current element if it does not exceed target
    if (target >= arr[i] && dp[i - 1][target - arr[i]]) {
        curr.push_back(arr[i]);
        findSubsets(arr, dp, i - 1, target - arr[i], curr, res);
        curr.pop_back();
    }
}
```

```
// Returns all subsets with the given target sum
vector<vector<int>> perfectSum(vector<int> &arr, int target) {
    int n = arr.size();
    if (n == 0 || target < 0)
        return {};

```

```
    // DP table to store subset sum possibilities
```

```

vector<vector<bool>> dp(n, vector<bool>(target + 1, false));

// Correct DP initialization for handling zeroes
dp[0][0] = true;
if (arr[0] <= target)
    dp[0][arr[0]] = true;

for (int i = 1; i < n; ++i) {
    for (int j = 0; j <= target; ++j) {
        dp[i][j] = dp[i - 1][j] || (arr[i] <= j && dp[i - 1][j - arr[i]]);
    }
}

// If no subsets sum to target, return empty
if (!dp[n - 1][target])
    return {};

vector<vector<int>> res;
vector<int> curr;
findSubsets(arr, dp, n - 1, target, curr, res);
return res;
}

// Function to print subsets in required format
void print2dArray(vector<vector<int>> &arr) {
    if (arr.empty()) {

        // No valid subsets found
        cout << "-1\n";
        return;
    }

    // Printing subsets in formatted output
    for (int row = 0; row < arr.size(); row++) {
        cout << "[";
        for (int col = arr[row].size() - 1; col >= 0; col--) {
            cout << arr[row][col];
            if (col != 0) {
                cout << ", ";
            }
        }
        cout << "];";
        if (row < arr.size() - 1)
            cout << ", ";
    }
}

// Driver function
int main() {
    vector<int> arr = {5, 2, 3, 10, 6, 8};
    int target = 10;
    vector<vector<int>> result = perfectSum(arr, target);
    print2dArray(result);
}

```

```
    return 0;
}
```

Regex Matching

```
#include <iostream>
#include <vector>
using namespace std;

bool isMatch(string t, string p) {
    int n = t.size();
    int m = p.size();

    // DP table where dp[i][j] means whether first i characters in t
    // match the first j characters in p
    vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));

    // Empty pattern matches empty text
    dp[0][0] = true;

    // Deals with patterns like a*, a*b*, a*b*c* etc, where '*'
    // can eliminate preceding character
    for (int j = 1; j <= m; ++j) {
        if (p[j - 1] == '*' && j > 1) {
            dp[0][j] = dp[0][j - 2];
        }
    }

    // Fill the table
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {

            // Characters match
            if (p[j - 1] == '.' || t[i - 1] == p[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1];
            }

            else if (p[j - 1] == '*' && j > 1) {

                // Two cases:
                // 1. '*' represents zero occurrence of the preceding character
                // 2. '*' represents one or more occurrence of the preceding character
                dp[i][j] = dp[i][j - 2] ||
                    (dp[i - 1][j] && (p[j - 2] == t[i - 1] || p[j - 2] == '.'));
            }
        }
    }
}
```

```

    return dp[n][m];
}

int main() {
    cout << boolalpha << isMatch("aab", "a.*") << endl;
    cout << boolalpha << isMatch("aa", "a") << endl;
    cout << boolalpha << isMatch("aa", "a*") << endl;
    cout << boolalpha << isMatch("ab", ".*") << endl;
    cout << boolalpha << isMatch("mississippi", "mis*is*p*.") << endl;
    return 0;
}

```

Subset Sum - Count

```

// C++ Program for Space Optimized Dynamic Programming
// Solution to Subset Sum Problem
#include <bits/stdc++.h>
using namespace std;

// Returns true if there is a subset of arr[]
// with sum equal to given sum
bool isSubsetSum(vector<int> arr, int sum) {
    int n = arr.size();
    vector<bool> prev(sum + 1, false), curr(sum + 1);

    // Mark prev[0] = true as it is true
    // to make sum = 0 using 0 elements
    prev[0] = true;

    // Fill the subset table in
    // bottom up manner
    for (int i = 1; i <= n; i++) {
        for (int j = 0; j <= sum; j++) {
            if (j < arr[i - 1])
                curr[j] = prev[j];
            else
                curr[j] = (prev[j] || prev[j - arr[i - 1]]);
        }
        prev = curr;
    }
    return prev[sum];
}

// Returns the count of subsets of arr[] with sum equal to given sum
int countSubsetSum(vector<int> arr, int sum) {
    int n = arr.size();

```

```
vector<int> prev(sum + 1, 0), curr(sum + 1);

// Base case: there's 1 way to make sum = 0 (empty subset)
prev[0] = 1;

for (int i = 1; i <= n; i++) {
    for (int j = 0; j <= sum; j++) {
        if (j < arr[i - 1]) {
            curr[j] = prev[j];
        } else {
            curr[j] = prev[j] + prev[j - arr[i - 1]];
        }
    }
    prev = curr;
}
return prev[sum];
}
```

Subset Sums

```
#include <bits/stdc++.h>
using namespace std;

// Prints sums of all subsets of array
void subsetSums(vector<int> &nums, int n) {
    // There are total 2^n subsets
    vector<int> s = {0}; // store the sums

    for (int i = 0; i < n; i++) {
        const int v = s.size();
        for (int t = 0; t < v; t++) {
            s.push_back(s[t] + nums[i]); // add this element with previous subsets
        }
    }
    // Print
    for (int i = 0; i < s.size(); i++)
        cout << s[i] << " ";
}
```

Graphs

Tarjan Strongly Connected Components

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 100005;
vector<int> adj[MAXN];
int disc[MAXN], low[MAXN], timer;
bool inStack[MAXN];
stack<int> st;
vector<vector<int>> sccs;

void tarjan(int u) {
    disc[u] = low[u] = ++timer;
    st.push(u);
    inStack[u] = true;

    for (int v : adj[u]) {
        if (!disc[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
        } else if (inStack[v]) {
            low[u] = min(low[u], disc[v]);
        }
    }

    if (disc[u] == low[u]) {
        vector<int> scc;
        int v;
        do {
            v = st.top();
            st.pop();
            inStack[v] = false;
            scc.push_back(v);
        } while (v != u);
        sccs.push_back(scc);
    }
}

void findSCCs(int n) {
    memset(disc, 0, sizeof disc);
    memset(low, 0, sizeof low);
    memset(inStack, false, sizeof inStack);
    while (!st.empty())
        st.pop();
    sccs.clear();
    timer = 0;
}
```

```

    for (int i = 0; i < n; ++i)
        if (!disc[i])
            tarjan(i);
}

int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    // Example usage
    int n = 5;
    // Add edges manually or via input
    adj[1].push_back(0);
    adj[0].push_back(2);
    adj[2].push_back(1);
    adj[0].push_back(3);
    adj[3].push_back(4);

    findSCCs(n);

    // Output SCCs
    for (auto &scc : sccs) {
        for (int x : scc)
            cout << x << " ";
        cout << "\n";
    }

    return 0;
}

```

Bipartite Graph Maximum Matching

```

// Note: If the matching found by the Hopcroft-Karp algorithm is not a perfect
// matching, it is guaranteed that the graph does not have one.

#define PROBLEM "https://judge.yosupo.jp/problem/bipartitematching"

#pragma GCC optimize("O3")
#pragma GCC optimize("unroll-loops")
#pragma GCC target("avx,avx2,sse,sse2,sse3,sse4,popcnt")

#include <bits/stdc++.h>

namespace IO {
constexpr bool UNSAFE = false;
constexpr int GLOB_BUF_SIZE = 1 << 15;

```

```

#ifndef DEBUG
#define CHANGE_DEFAULT_STREAMS
static struct FastInput {
    FastInput() {
        if constexpr (UNSAFE) {
            chars_read = fread(buf, 1, BUF_SIZE, in);
            buf_pos = 0;
            buf[0] = (chars_read == 0 ? -1 : buf[0]);
        }
    }
    static constexpr int BUF_SIZE = GLOB_BUF_SIZE;
    char buf[BUF_SIZE];
    size_t chars_read = 0;
    size_t buf_pos = 0;
    FILE *in = stdin;
    char cur = 0;
    inline char get_char() {
        if constexpr (!UNSAFE) {
            if (buf_pos >= chars_read) {
                chars_read = fread(buf, 1, BUF_SIZE, in);
                buf_pos = 0;
                buf[0] = (chars_read == 0 ? -1 : buf[0]);
            }
        }
        return cur = buf[buf_pos++];
    }
    template <typename T> inline FastInput *tie(T) { return this; }
    inline void sync_with_stdio(bool) {}
    inline explicit operator bool() { return cur != -1; }
    inline static bool is_blank(char c) { return c <= ' '; }
    inline bool skip_blanks() {
        while (is_blank(cur) && cur != -1)
            get_char();
        return cur != -1;
    }
    inline FastInput &operator>>(char &c) {
        skip_blanks();
        c = cur;
        return *this;
    }
    inline FastInput &operator>>(std::string &s) {
        if (skip_blanks()) {
            s.clear();
            do {
                s += cur;
            } while (!is_blank(get_char()));
        }
        return *this;
    }
    template <typename T> inline FastInput &read_integer(T &n) {
        // unsafe, doesn't check that characters are actually digits
        n = 0;
        if (skip_blanks()) {
            int sign = +1;

```

```

        if (cur == '-') {
            sign = -1;
            get_char();
        }
        do {
            n += n + (n << 3) + cur - '0';
        } while (!is_blank(get_char()));
        n *= sign;
    }
    return *this;
}

template <typename T>
inline typename std::enable_if<std::is_integral<T>::value, FastInput &>::type
operator>>(T &n) {
    return read_integer(n);
}

#ifdef _WIN32 || defined(_WIN64)
inline FastInput &operator>>(__int128 &n) { return read_integer(n); }
#endif

template <typename T>
inline typename std::enable_if<std::is_floating_point<T>::value,
                               FastInput &>::type
operator>>(T &n) {
    // not sure if really fast, for compatibility only
    n = 0;
    if (skip_blanks()) {
        std::string s;
        (*this) >> s;
        sscanf(s.c_str(), "%lf", &n);
    }
    return *this;
}

} fast_input;

#define cin IO::fast_input
static struct FastOutput {
    static constexpr int BUF_SIZE = GLOB_BUF_SIZE;
    char buf[BUF_SIZE];
    size_t buf_pos = 0;
    static constexpr int TMP_SIZE = GLOB_BUF_SIZE;
    char tmp[TMP_SIZE];
    FILE *out = stdout;
    inline void put_char(char c) {
        buf[buf_pos++] = c;
        if (buf_pos == BUF_SIZE) {
            fwrite(buf, 1, buf_pos, out);
            buf_pos = 0;
        }
    }
} ~FastOutput() { fwrite(buf, 1, buf_pos, out); }
inline FastOutput &operator<<(char c) {
    put_char(c);
    return *this;
}

inline FastOutput &operator<<(const char *s) {

```

```

    while (*s)
        put_char(*s++);
    return *this;
}

inline FastOutput &operator<<(const std::string &s) {
    for (auto x : s)
        put_char(x);
    return *this;
}

template <typename T> inline char *integer_to_string(T n) {
    // beware of TMP_SIZE
    char *p = tmp + TMP_SIZE - 1;
    if (n == 0)
        *--p = '0';
    else {
        bool is_negative = false;
        if (n < 0) {
            is_negative = true;
            n = -n;
        }
        while (n > 0) {
            *--p = (char)('0' + n % 10);
            n /= 10;
        }
        if (is_negative)
            *--p = '-';
    }
    return p;
}

template <typename T>
inline typename std::enable_if<std::is_integral<T>::value, char *>::type
stringify(T n) {
    return integer_to_string(n);
}

#ifdef _WIN32 || defined(_WIN64)
    inline char *stringify(__int128 n) { return integer_to_string(n); }
#endif

template <typename T>
inline typename std::enable_if<std::is_floating_point<T>::value, char *>::type
stringify(T n) {
    sprintf(tmp, "%.17f", n);
    return tmp;
}

template <typename T> inline FastOutput &operator<<(const T &n) {
    auto p = stringify(n);
    for (; *p != 0; p++)
        put_char(*p);
    return *this;
}

} fast_output;
#define cout IO::fast_output
#endif
} // namespace IO

```

```

struct Random : std::mt19937 {
    using std::mt19937::mt19937;
    using std::mt19937::operator();
    static int64_t gen_seed() {
        return std::chrono::steady_clock::now().time_since_epoch().count();
    }
    Random() : std::mt19937(gen_seed()) {}
    template <class Int>
    auto operator()(Int a, Int b)
        -> std::enable_if_t<std::is_integral_v<Int>, Int> {
        return std::uniform_int_distribution<Int>(a, b)(*this);
    }
    template <class Int>
    auto operator()(Int a) -> std::enable_if_t<std::is_integral_v<Int>, Int> {
        return std::uniform_int_distribution<Int>(0, a - 1)(*this);
    }
    template <class Real>
    auto operator()(Real a, Real b)
        -> std::enable_if_t<std::is_floating_point_v<Real>, Real> {
        return std::uniform_real_distribution<Real>(a, b)(*this);
    }
};

template <bool ToShuffle = false> struct bipartite_matching {
    int n_left, n_right, flow = 0;
    std::vector<std::vector<int>> g;
    std::vector<int> match_from_left, match_from_right;

    bipartite_matching(int _n_left, int _n_right)
        : n_left(_n_left), n_right(_n_right), g(_n_left),
          match_from_left(_n_left, -1), match_from_right(_n_right, -1),
          dist(_n_left) {}

    void add(int u, int v) { g[u].push_back(v); }

    std::vector<int> dist;

    void bfs() {
        std::queue<int> q;
        for (int u = 0; u < n_left; ++u) {
            if (!match_from_left[u])
                q.push(u), dist[u] = 0;
            else
                dist[u] = -1;
        }
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto v : g[u])
                if (~match_from_right[v] && !~dist[match_from_right[v]]) {
                    dist[match_from_right[v]] = dist[u] + 1;
                    q.push(match_from_right[v]);
                }
        }
    }
};

```

```

}

bool dfs(int u) {
    for (auto v : g[u])
        if (!~match_from_right[v]) {
            match_from_left[u] = v, match_from_right[v] = u;
            return true;
        }
    for (auto v : g[u])
        if (dist[match_from_right[v]] == dist[u] + 1 &&
            dfs(match_from_right[v])) {
            match_from_left[u] = v, match_from_right[v] = u;
            return true;
        }
    return false;
}

int get_max_matching() {
    if constexpr (ToShuffle) {
        Random rng;
        for (int i = 0; i < n_left; ++i)
            std::random_shuffle(std::begin(g[i]), std::end(g[i]), rng);
    }
    while (true) {
        bfs();
        int augment = 0;
        for (int u = 0; u < n_left; ++u)
            if (!~match_from_left[u])
                augment += dfs(u);
        if (!augment)
            break;
        flow += augment;
    }
    return flow;
}

std::pair<std::vector<int>, std::vector<int>> minimum_vertex_cover() {
    std::vector<int> L, R;
    for (int u = 0; u < n_left; ++u) {
        if (!~dist[u])
            L.push_back(u);
        else if (~match_from_left[u])
            R.push_back(match_from_left[u]);
    }
    return {L, R};
}

std::vector<std::pair<int, int>> get_edges() {
    std::vector<std::pair<int, int>> ans;
    for (int u = 0; u < n_left; ++u)
        if (match_from_left[u] != -1)
            ans.emplace_back(u, match_from_left[u]);
    return ans;
}

```

```
};

int main() {
    int n_left, n_right, m;
    cin >> n_left >> n_right >> m;
    bipartite_matching<false> matching(n_left, n_right);
    while (m--) {
        int u, v;
        cin >> u >> v;
        matching.add(u, v);
    }
    cout << matching.get_max_matching() << '\n';
    for (auto [u, v] : matching.get_edges())
        cout << u << ' ' << v << '\n';
}
```

Eulerian Cycle Or Path

```
// C++ program to check whether a graph is
// Eulerian Path, Eulerian Circuit, or neither
#include <bits/stdc++.h>
using namespace std;

// DFS to check connectivity, excluding zero-degree vertices
void dfs(int node, vector<int> adj[], vector<bool> &visited) {
    visited[node] = true;

    for (int neighbor : adj[node]) {
        if (!visited[neighbor]) {
            dfs(neighbor, adj, visited);
        }
    }
}

// Function to check Eulerian Path or Circuit
// 2 if Cycle, 1 if Path, 0 if neither
int isEulerCircuit(int v, vector<int> adj[]) {
    vector<bool> visited(v, false);

    // Find first vertex with non-zero degree
    int start = -1;
    for (int i = 0; i < v; i++) {
        if (adj[i].size() > 0) {
            start = i;
            break;
        }
    }
}
```

```
// No edges: graph is trivially Eulerian
if (start == -1) {
    return 2;
}

// DFS from the first non-zero degree vertex
dfs(start, adj, visited);

// Check if all non-zero degree vertices are connected
for (int i = 0; i < v; i++) {
    if (adj[i].size() > 0 && !visited[i]) {
        return 0; // Not connected
    }
}

// Count vertices with odd degree
int odd = 0;
for (int i = 0; i < v; i++) {
    if (adj[i].size() % 2 != 0) {
        odd++;
    }
}

// Apply Eulerian rules
if (odd == 0) {
    return 2;
} else if (odd == 2) {
    return 1;
} else {
    return 0;
}
}

// Driver code
int main() {

    int v = 5;
    vector<int> adj[5] = {{1, 2, 3}, {0, 2}, {1, 0}, {0, 4}, {3}};

    cout << isEulerCircuit(v, adj);

    return 0;
}
```

Hamiltonian Path

```

// C++ program for the above approach

#include <bits/stdc++.h>
using namespace std;
const int N = 5;

// Function to check whether there
// exists a Hamiltonian Path or not
bool Hamiltonian_path(vector<vector<int>> &adj, int N) {
    int dp[N][(1 << N)];

    // Initialize the table
    memset(dp, 0, sizeof dp);

    // Set all dp[i][(1 << i)] to
    // true
    for (int i = 0; i < N; i++)
        dp[i][(1 << i)] = true;

    // Iterate over each subset
    // of nodes
    for (int i = 0; i < (1 << N); i++) {

        for (int j = 0; j < N; j++) {

            // If the jth nodes is included
            // in the current subset
            if (i & (1 << j)) {

                // Find K, neighbour of j
                // also present in the
                // current subset
                for (int k = 0; k < N; k++) {

                    if (i & (1 << k) && adj[k][j] && j != k && dp[k][i ^ (1 << j)]) {

                        // Update dp[j][i]
                        // to true
                        dp[j][i] = true;
                        break;
                    }
                }
            }
        }
    }

    // Traverse the vertices
    for (int i = 0; i < N; i++) {

        // Hamiltonian Path exists
        if (dp[i][(1 << N) - 1])
            return true;
    }
}

```

```

    // Otherwise, return false
    return false;
}

// Driver Code
int main() {

    // Input
    vector<vector<int>> adj = {
        {0, 1, 1, 1, 0}, {1, 0, 1, 0, 1}, {1, 1, 0, 1, 1}, {1, 0, 1, 0, 0}};
    int N = adj.size();

    // Function Call
    if (Hamiltonian_path(adj, N))
        cout << "YES";
    else
        cout << "NO";

    return 0;
}

```

Hungarian

```

#include <cassert>
#include <iostream>
#include <limits>
#include <vector>

using namespace std;

template <typename T, typename U> using Pair = std::pair<T, U>;
template <typename T> using Vector = std::vector<T>;

template <typename T> using NumericLimits = std::numeric_limits<T>;

/**
 * @brief Checks if b < a
 *
 * Sets a = min(a, b)
 * @param a The first parameter to check
 * @param b The second parameter to check
 * @tparam The type to perform the check on
 * @return true if b < a
 */
template <typename T> constexpr bool ckmin(T &a, const T &b) {
    return b < a ? a = b, true : false;
}

```

```

}

/**
 * @brief Performs the Hungarian algorithm.
 *
 * Given J jobs and W workers (J <= W), computes the minimum cost to assign each
 * prefix of jobs to distinct workers.
 *
 * @tparam T a type large enough to represent integers on the order of J *
 * max(|C|)
 * @param C a matrix of dimensions JxW such that C[j][w] = cost to assign j-th
 * job to w-th worker (possibly negative)
 *
 * @return a vector of length J, with the j-th entry equaling the minimum cost
 * to assign the first (j+1) jobs to distinct workers
 */
template <typename T> Vector<T> hungarian(const Vector<Vector<T>> &C) {
    const int J = static_cast<int>(C.size());
    const int W = static_cast<int>(C[0].size());
    assert(J <= W);
    // job[w] = job assigned to w-th worker, or -1 if no job assigned
    // note: a W-th worker was added for convenience
    Vector<int> job(W + 1, -1);
    Vector<T> ys(J);
    Vector<T> yt(W + 1); // potentials
    // -yt[W] will equal the sum of all deltas
    Vector<T> answers;
    const T inf = NumericLimits<T>::max();
    for (int jCur = 0; jCur < J; ++jCur) { // assign jCur-th job
        int wCur = W;
        job[wCur] = jCur;
        // min reduced cost over edges from Z to worker w
        Vector<T> minTo(W + 1, inf);
        Vector<int> prev(W + 1, -1); // previous worker on alternating path
        Vector<bool> inZ(W + 1);    // whether worker is in Z
        while (job[wCur] != -1) {    // runs at most jCur + 1 times
            inZ[wCur] = true;
            const int j = job[wCur];
            T delta = inf;
            int wNext;
            for (int w = 0; w < W; ++w) {
                if (!inZ[w]) {
                    if (ckmin(minTo[w], C[j][w] - ys[j] - yt[w]))
                        prev[w] = wCur;
                    if (ckmin(delta, minTo[w]))
                        wNext = w;
                }
            }
        }
        // delta will always be nonnegative,
        // except possibly during the first time this loop runs
        // if any entries of C[jCur] are negative
        for (int w = 0; w <= W; ++w) {
            if (inZ[w]) {
                ys[job[w]] += delta;
            }
        }
    }
}

```

```

        yt[w] -= delta;
    } else {
        minTo[w] -= delta;
    }
}
wCur = wNext;
}
// update assignments along alternating path
for (int w; wCur != W; wCur = w)
    job[wCur] = job[w = prev[wCur]];
answers.push_back(-yt[W]);
}
return answers;
}

/**
 * @brief Performs a sanity check for the Hungarian algorithm.
 *
 * Sanity check: https://en.wikipedia.org/wiki/Hungarian\_algorithm#Example
 * First job (5):
 *   clean bathroom: Bob -> 5
 * First + second jobs (9):
 *   clean bathroom: Bob -> 5
 *   sweep floors: Alice -> 4
 * First + second + third jobs (15):
 *   clean bathroom: Alice -> 8
 *   sweep floors: Carol -> 4
 *   wash windows: Bob -> 3
 */
void sanityCheckHungarian() {
    Vector<Vector<int>> costs{{8, 5, 9}, {4, 2, 4}, {7, 3, 8}};
    assert((hungarian(costs) == Vector<int>{5, 9, 15}));
    std::println(stderr, "Sanity check passed.");
}

/**
 * @brief solves https://open.kattis.com/problems/cordonbleu
 */
void cordonBleu() {
    int N;
    int M;
    std::cin >> N >> M;
    Vector<Pair<int, int>> B(N);
    Vector<Pair<int, int>> C(M);
    Vector<Pair<int, int>> bottles(N);
    Vector<Pair<int, int>> couriers(M);
    for (const Pair<int, int> &b : bottles)
        std::cin >> b.first >> b.second;
    for (const Pair<int, int> &c : couriers)
        std::cin >> c.first >> c.second;
    Pair<int, int> rest;
    std::cin >> rest.first >> rest.second;
    Vector<Vector<int>> costs(N, Vector<int>(N + M - 1));
    auto dist = [&](const Pair<int, int> &x, const Pair<int, int> &y) -> int {

```

```

    return std::abs(x.first - y.first) + std::abs(x.second - y.second);
};
for (int b = 0; b < N; ++b) {
    for (int c = 0; c < M; ++c) // courier -> bottle -> restaurant
        costs[b][c] = dist(couriers[c], bottles[b]) + dist(bottles[b], rest);
    for (int _ = 0; _ < N - 1; ++_) // restaurant -> bottle -> restaurant
        costs[b][_ + M] = 2 * dist(bottles[b], rest);
}
std::println(hungarian(costs).back());
}

/**
 * @brief Entry point into the program.
 *
 * @return The return code of the program.
 */
int main() {
    sanityCheckHungarian();
    cordonBleu();
}

```

Hungarian-Shortest-Paths

```

#include <cassert>
#include <limits>
#include <vector>

using namespace std;

// The implementation of the Hungarian algorithm can be refactored to highlight
// this connection. The worker potentials, represented by h in this new
// approach, are equivalent to the potentials y from the previous solution, with
// only a constant offset. For cases where the graph is sparse (meaning there
// are only M allowed job-worker pairs), you can optimize the algorithm to run
// in  $O(J \cdot M + J^2 \cdot \log(W))$  time. This is achieved by using a Fibonacci heap to find
// the next worker to process, rather than iterating through all W workers. This
// modification significantly improves performance by efficiently identifying
// the worker with the minimum distance.

template <typename T, typename U> using Pair = std::pair<T, U>;
template <typename T> using Vector = std::vector<T>;

template <typename T> using NumericLimits = std::numeric_limits<T>;

/**
 * @brief Checks if b < a
 *

```

```

* Sets a = min(a, b)
* @param a The first parameter to check
* @param b The second parameter to check
* @tparam The type to perform the check on
* @return true if b < a
*/
template <typename T> constexpr bool ckmin(T &a, const T &b) {
    return b < a ? a = b, true : false;
}

template <typename T> Vector<T> hungarian(const Vector<Vector<T>> &C) {
    const int J = static_cast<int>(C.size());
    const int W = static_cast<int>(C[0].size());
    assert(J <= W);
    // job[w] = job assigned to w-th worker, or -1 if no job assigned
    // note: a W-th worker was added for convenience
    Vector<int> job(W + 1, -1);
    Vector<T> h(W); // Johnson potentials
    Vector<T> answers;
    T ansCur = 0;
    const T inf = std::numeric_limits<T>::max();
    // assign jCur-th job using Dijkstra with potentials
    for (int jCur = 0; jCur < J; ++jCur) {
        int wCur = W; // unvisited worker with minimum distance
        job[wCur] = jCur;
        Vector<T> dist(W + 1, inf); // Johnson-reduced distances
        dist[W] = 0;
        Vector<bool> vis(W + 1); // whether visited yet
        Vector<int> prev(W + 1, -1); // previous worker on shortest path
        while (job[wCur] != -1) { // Dijkstra step: pop min worker from heap
            T minDist = inf;
            vis[wCur] = true;
            int wNext = -1; // next unvisited worker with minimum distance
            // consider extending shortest path by wCur -> job[wCur] -> w
            for (int w = 0; w < W; ++w) {
                if (!vis[w]) {
                    // sum of reduced edge weights wCur -> job[wCur] -> w
                    T edge = C[job[wCur]][w] - h[w];
                    if (wCur != W) {
                        edge -= C[job[wCur]][wCur] - h[wCur];
                        assert(edge >= 0); // consequence of Johnson potentials
                    }
                    if (ckmin(dist[w], dist[wCur] + edge))
                        prev[w] = wCur;
                    if (ckmin(minDist, dist[w]))
                        wNext = w;
                }
            }
            wCur = wNext;
        }
        for (int w = 0; w < W; ++w) { // update potentials
            ckmin(dist[w], dist[wCur]);
            h[w] += dist[w];
        }
    }
}

```

```

    ansCur += h[wCur];
    for (int w; wCur != W; wCur = w)
        job[wCur] = job[w = prev[wCur]];
    answers.push_back(ansCur);
}
return answers;
}

```

Johnsons ALogrithm

```

#include <iostream>
#include <limits>
#include <vector>

#define INF std::numeric_limits<int>::max()

using namespace std;

// Function to find the vertex with the minimum distance
// that has not yet been included in the shortest path tree
int Min_Distance(const vector<int> &dist, const vector<bool> &visited) {
    int min = INF, min_index;
    for (int v = 0; v < dist.size(); ++v) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}

// Function to perform Dijkstra's algorithm on the modified graph
void Dijkstra_Algorithm(const vector<vector<int>> &graph,
                        const vector<vector<int>> &altered_graph, int source) {
    int V = graph.size(); // Number of vertices
    vector<int> dist(V, INF); // Distance from source to each vertex
    vector<bool> visited(V, false); // Track visited vertices

    dist[source] = 0; // Distance to source itself is 0

    // Compute shortest path for all vertices
    for (int count = 0; count < V - 1; ++count) {
        // Select the vertex with the minimum distance that hasn't been visited
        int u = Min_Distance(dist, visited);
        visited[u] = true; // Mark this vertex as visited

        // Update the distance value of the adjacent vertices of the selected vertex
    }
}

```

```

    for (int v = 0; v < V; ++v) {
        if (!visited[v] && graph[u][v] != 0 && dist[u] != INF &&
            dist[u] + altered_graph[u][v] < dist[v]) {
            dist[v] = dist[u] + altered_graph[u][v];
        }
    }
}

// Print the shortest distances from the source
cout << "Shortest Distance from vertex " << source << ":\n";
for (int i = 0; i < V; ++i) {
    cout << "Vertex " << i << ": "
        << (dist[i] == INF ? "INF" : to_string(dist[i])) << endl;
}
}

// Function to perform Bellman-Ford algorithm to find shortest distances
// from a source vertex to all other vertices
vector<int> BellmanFord_Algorithm(const vector<vector<int>> &edges, int V) {
    vector<int> dist(V + 1, INF); // Distance from source to each vertex
    dist[V] = 0; // Distance to the new source vertex (added vertex) is 0

    // Add a new source vertex to the graph and connect it to all original
    // vertices with 0 weight edges
    vector<vector<int>> edges_with_extra(edges);
    for (int i = 0; i < V; ++i) {
        edges_with_extra.push_back({V, i, 0});
    }

    // Relax all edges |V| - 1 times
    for (int i = 0; i < V; ++i) {
        for (const auto &edge : edges_with_extra) {
            if (dist[edge[0]] != INF && dist[edge[0]] + edge[2] < dist[edge[1]]) {
                dist[edge[1]] = dist[edge[0]] + edge[2];
            }
        }
    }

    return vector<int>(dist.begin(),
                      dist.begin() +
                      V); // Return distances excluding the new source vertex
}

// Function to implement Johnson's Algorithm
void JohnsonAlgorithm(const vector<vector<int>> &graph) {
    int V = graph.size(); // Number of vertices
    vector<vector<int>> edges;

    // Collect all edges from the graph
    for (int i = 0; i < V; ++i) {
        for (int j = 0; j < V; ++j) {
            if (graph[i][j] != 0) {
                edges.push_back({i, j, graph[i][j]});
            }
        }
    }
}

```

```

    }

    // Get the modified weights from Bellman-Ford algorithm
    vector<int> altered_weights = BellmanFord_Algorithm(edges, V);
    vector<vector<int>> altered_graph(V, vector<int>(V, 0));

    // Modify the weights of the edges to remove negative weights
    for (int i = 0; i < V; ++i) {
        for (int j = 0; j < V; ++j) {
            if (graph[i][j] != 0) {
                altered_graph[i][j] =
                    graph[i][j] + altered_weights[i] - altered_weights[j];
            }
        }
    }

    // Print the modified graph with re-weighted edges
    cout << "Modified Graph:\n";
    for (const auto &row : altered_graph) {
        for (int weight : row) {
            cout << weight << ' ';
        }
        cout << endl;
    }

    // Run Dijkstra's algorithm for every vertex as the source
    for (int source = 0; source < V; ++source) {
        cout << "\nShortest Distance with vertex " << source << " as the source:\n";
        Dijkstra_Algorithm(graph, altered_graph, source);
    }
}

// Main function to test the Johnson's Algorithm implementation
int main() {
    // Define a graph with possible negative weights
    vector<vector<int>> graph = {
        {0, -5, 2, 3}, {0, 0, 4, 0}, {0, 0, 0, 1}, {0, 0, 0, 0}};

    // Execute Johnson's Algorithm
    JohnsonAlgorithm(graph);
    return 0;
}

```

Johnsons ALogrithm Scheduling

```
#include <bits/stdc++.h>
```

```

using namespace std;
using ll = long long;
using vi = vector<int>;
using vll = vector<ll>;

#define REP(i, a, b) for (int i = (a); i < (b); ++i)
#define ALL(x) (x).begin(), (x).end()

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N;
    if (!(cin >> N))
        return 0;
    vector<ll> p1(N), p2(N);
    for (int i = 0; i < N; ++i)
        cin >> p1[i] >> p2[i];

    // Johnson's algorithm
    vector<int> A(N);
    iota(A.begin(), A.end(), 0); // remaining job indices
    deque<int> L2;
    vector<int> L1;
    vector<char> used(N, 0);

    int remaining = N;
    while (remaining > 0) {
        // find min among available p1[i], p2[i] where not used
        ll mn = LLONG_MAX;
        int which = -1; // job index
        int side = 0; // 1 => belongs to p1, 2 => belongs to p2
        for (int i = 0; i < N; ++i) {
            if (used[i])
                continue;
            if (p1[i] < mn) {
                mn = p1[i];
                which = i;
                side = 1;
            }
            if (p2[i] < mn) {
                mn = p2[i];
                which = i;
                side = 2;
            }
        }
        // tie-breaking: above loop picks first found minimum (smallest index)
        if (which == -1)
            break;
        used[which] = 1;
        --remaining;
        if (side == 1)
            L1.push_back(which);
        else
    
```

```

    L2.push_front(which);
}

// final sequence: L1 followed by L2
vector<int> seq;
seq.reserve(N);
for (int x : L1)
    seq.push_back(x);
for (int x : L2)
    seq.push_back(x);

// simulate to compute makespan and optional completion times
ll t1 = 0, t2 = 0;
for (int idx : seq) {
    t1 += p1[idx]; // finish time on M1 for this job
    if (t2 < t1)
        t2 = t1; // M2 can only start after M1 finishes for that job and after M2
                  // is free
    t2 += p2[idx]; // finish time on M2 for this job
}
ll makespan = t2;

// Output: sequence (1-based indices) and makespan
for (int i = 0; i < (int)seq.size(); ++i) {
    if (i)
        cout << ' ';
    cout << (seq[i] + 1);
}
cout << '\n' << makespan << '\n';
return 0;
}

```

Maximal Independent Set

```

#include <iostream>
#include <vector>
using namespace std;

const int MAXN = 1005; // Adjust based on constraints
bool adj[MAXN][MAXN];
bool active[MAXN];

vector<int> findMIS(int n) {
    vector<int> mis;

    for (int i = 0; i < n; i++) {
        if (!active[i])

```

```
        continue;

    mis.push_back(i);
    active[i] = false;

    // Remove all neighbors of i
    for (int j = 0; j < n; j++) {
        if (adj[i][j]) {
            active[j] = false;
        }
    }
}

return mis;
}

int main() {
    int n, m;
    cin >> n >> m;

    // Initialize adjacency matrix
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u][v] = adj[v][u] = true;
    }

    // Initialize all nodes as active
    for (int i = 0; i < n; i++) {
        active[i] = true;
    }

    vector<int> mis = findMIS(n);

    // Output the result
    cout << mis.size() << endl;
    for (int node : mis) {
        cout << node << " ";
    }
    cout << endl;

    return 0;
}
```

Minimum Clique Cover

```

#include <bits/stdc++.h>
using namespace std;

int n;
vector<vector<int>> g;

vector<int> match, p, base;
vector<char> used, blossom;
vector<int> q;

int lca(int a, int b) {
    vector<char> used_local(n, false);
    while (true) {
        a = base[a];
        used_local[a] = true;
        if (match[a] == -1)
            break;
        a = p[match[a]];
    }
    while (true) {
        b = base[b];
        if (used_local[b])
            return b;
        b = p[match[b]];
    }
}

void mark_path(int v, int b, int children) {
    while (base[v] != b) {
        blossom[base[v]] = blossom[base[match[v]]] = true;
        p[v] = children;
        children = match[v];
        v = p[match[v]];
    }
}

int find_path(int root) {
    used.assign(n, false);
    p.assign(n, -1);
    for (int i = 0; i < n; ++i)
        base[i] = i;

    q.clear();
    q.push_back(root);
    used[root] = true;
    size_t qh = 0;
    while (qh < q.size()) {
        int v = q[qh++];
        for (int to : g[v]) {
            if (base[v] == base[to] || match[v] == to)
                continue;
            if (to == root || (match[to] != -1 && p[match[to]] != -1)) {

```

```

    int cur = lca(v, to);
    blossom.assign(n, false);
    mark_path(v, cur, to);
    mark_path(to, cur, v);
    for (int i = 0; i < n; ++i) {
        if (blossom[base[i]]) {
            base[i] = cur;
            if (!used[i]) {
                used[i] = true;
                q.push_back(i);
            }
        }
    }
} else if (p[to] == -1) {
    p[to] = v;
    if (match[to] == -1)
        return to;
    int nxt = match[to];
    used[nxt] = true;
    q.push_back(nxt);
}
}
return -1;
}

void augment_path(int finish) {
    int v = finish;
    while (v != -1) {
        int pv = p[v];
        int nv = (pv == -1 ? -1 : match[pv]);
        match[v] = pv;
        if (pv != -1)
            match[pv] = v;
        v = nv;
    }
}

int max_matching() {
    match.assign(n, -1);
    base.resize(n);
    int res = 0;
    for (int i = 0; i < n; ++i) {
        if (match[i] == -1) {
            int v = find_path(i);
            if (v == -1)
                continue;
            ++res;
            augment_path(v);
        }
    }
    return res;
}

```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    if (!(cin >> n >> m))
        return 0;
    g.assign(n, {});
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        // assume input is 0-indexed. If input is 1-indexed, subtract 1 here.
        if (u == v)
            continue;
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int matching_size = max_matching();
    int min_clique_cover = n - matching_size;
    cout << min_clique_cover << '\n';

    vector<char> used_out(n, false);
    // print matched pairs (each pair once) and singletons
    for (int v = 0; v < n; ++v) {
        if (match[v] != -1 && v < match[v]) {
            cout << 2 << ' ' << v << ' ' << match[v] << '\n';
            used_out[v] = used_out[match[v]] = true;
        }
    }
    for (int v = 0; v < n; ++v) {
        if (!used_out[v]) {
            cout << 1 << ' ' << v << '\n';
        }
    }

    return 0;
}
```

Repository Code Listing

BinaryHeap.cpp

```
1  #include "BinaryHeap.h"
2
3  void BinaryHeap::Clear()
4  {
5      key.clear();
6      pos.clear();
7      satellite.clear();
8  }
9
10 void BinaryHeap::Insert(double k, int s)
11 {
12     //Adjust the structures to fit new data
13     if(s >= (int)pos.size())
14     {
15         pos.resize(s+1, -1);
16         key.resize(s+1);
17         //Recall that position 0 of satellite is unused
18         satellite.resize(s+2);
19     }
20     //If satellite is already in the heap
21     else if(pos[s] != -1)
22     {
23         throw "Error: satellite already in heap";
24     }
25
26     int i;
27     for(i = ++size; i/2 > 0 && GREATER(key[satellite[i/2]], k); i /=
28 2)
29     {
30         satellite[i] = satellite[i/2];
31         pos[satellite[i]] = i;
32     }
33     satellite[i] = s;
34     pos[s] = i;
35     key[s] = k;
36 }
37
38 int BinaryHeap::Size()
39 {
40     return size;
41 }
42
43 int BinaryHeap::DeleteMin()
44 {
45     if(size == 0)
46         throw "Error: empty heap";
47
48     int min = satellite[1];
49     int slast = satellite[size--];
50
51
52     int child;
53     int i;
54     for(i = 1, child = 2; child <= size; i = child, child *= 2)
```

```

    {
55         if(child < size && GREATER(key[satellite[child]],
56 key[satellite[child+1]]))
57             child++;
58
59         if(GREATER(key[slast], key[satellite[child]]))
60         {
61             satellite[i] = satellite[child];
62             pos[satellite[child]] = i;
63         }
64         else
65             break;
66     }
67     satellite[i] = slast;
68     pos[slast] = i;
69
70     pos[min] = -1;
71
72     return min;
73 }
74
75 void BinaryHeap::ChangeKey(double k, int s)
76 {
77     Remove(s);
78     Insert(k, s);
79 }
80
81 void BinaryHeap::Remove(int s)
82 {
83     int i;
84     for(i = pos[s]; i/2 > 0; i /= 2)
85     {
86         satellite[i] = satellite[i/2];
87         pos[satellite[i]] = i;
88     }
89     satellite[1] = s;
90     pos[s] = 1;
91
92     DeleteMin();
93 }

```

BinaryHeap.h

```

1  #pragma once
2
3  #include "Globals.h"
4  #include <vector>
5  using namespace std;
6
7  /*
8  This is a binary heap for pairs of the type (double key, int satellite)
9  It is assumed that satellites are unique integers
10 This is the case with graph algorithms, in which satellites are vertex or
11 edge indices
12 */
13 class BinaryHeap
14 {
15 public:
16     BinaryHeap(): satellite(1), size(0) {};
17

```

```

18         //Inserts (key k, satellite s) in the heap
19         void Insert(double k, int s);
20         //Deletes the element with minimum key and returns its satellite
21         information
22         int DeleteMin();
23         //Changes the key of the element with satellite s
24         void ChangeKey(double k, int s);
25         //Removes the element with satellite s
26         void Remove(int s);
27         //Returns the number of elements in the heap
28         int Size();
29         //Resets the structure
30         void Clear();
31     private:
32         vector<double> key;//Given the satellite, this is its key
33         vector<int> pos;//Given the satellite, this is its position in
34         the heap
35         vector<int> satellite;//This is the heap!
36
37         //Number of elements in the heap
38         int size;
39     };

```

Example.cpp

```

1  #include "Matching.h"
2  #include <fstream>
3  #include "Graph.h"
4  #include <cstdlib>
5  #include <iostream>
6  #include <string>
7  #include <sstream>
8  using namespace std;
9
10 pair< Graph, vector<double> > CreateRandomGraph()
11 {
12     //random seed
13     int x;
14     cin >> x;
15     srand( x );
16
17     //Please see Graph.h for a description of the interface
18     int n = 50;
19
20     Graph G(n);
21     vector<double> cost;
22     for(int i = 0; i < n; i++)
23         for(int j = i+1; j < n; j++)
24             if(rand()%10 == 0)
25             {
26                 G.AddEdge(i, j);
27                 cost.push_back(rand()%1000);
28             }
29
30     return make_pair(G, cost);
31 }
32
33 Graph ReadGraph(string filename)
34 {

```

```

35         //Please see Graph.h for a description of the interface
36
37         ifstream file;
38         file.open(filename.c_str());
39
40         string s;
41         getline(file, s);
42         stringstream ss(s);
43         int n;
44         ss >> n;
45         getline(file, s);
46         ss.str(s);
47         ss.clear();
48         int m;
49         ss >> m;
50
51         Graph G(n);
52         for(int i = 0; i < m; i++)
53         {
54             getline(file, s);
55             ss.str(s);
56             ss.clear();
57             int u, v;
58             ss >> u >> v;
59
60             G.AddEdge(u, v);
61         }
62
63         file.close();
64         return G;
65     }
66
67     pair< Graph, vector<double> > ReadWeightedGraph(string filename)
68     {
69         //Please see Graph.h for a description of the interface
70
71         ifstream file;
72         file.open(filename.c_str());
73
74         string s;
75         getline(file, s);
76         stringstream ss(s);
77         int n;
78         ss >> n;
79         getline(file, s);
80         ss.str(s);
81         ss.clear();
82         int m;
83         ss >> m;
84
85         Graph G(n);
86         vector<double> cost(m);
87         for(int i = 0; i < m; i++)
88         {
89             getline(file, s);
90             ss.str(s);
91             ss.clear();
92             int u, v;
93             double c;
94             ss >> u >> v >> c;
95
96             G.AddEdge(u, v);
97             cost[G.GetEdgeIndex(u, v)] = c;
98         }

```

```

99
100     file.close();
101     return make_pair(G, cost);
102 }
103
104 void MinimumCostPerfectMatchingExample(string filename)
105 {
106     Graph G;
107     vector<double> cost;
108
109     //Read the graph
110     pair< Graph, vector<double> > p = ReadWeightedGraph(filename);
111     //pair< Graph, vector<double> > p = CreateRandomGraph();
112     G = p.first;
113     cost = p.second;
114
115     //Create a Matching instance passing the graph
116     Matching M(G);
117
118     //Pass the costs to solve the problem
119     pair< list<int>, double > solution =
120 M.SolveMinimumCostPerfectMatching(cost);
121
122     list<int> matching = solution.first;
123     double obj = solution.second;
124
125     cout << "Optimal matching cost: " << obj << endl;
126     cout << "Edges in the matching:" << endl;
127     for(list<int>::iterator it = matching.begin(); it !=
128 matching.end(); it++)
129     {
130         pair<int, int> e = G.GetEdge( *it );
131
132         cout << e.first << " " << e.second << endl;
133     }
134 }
135
136 void MaximumMatchingExample(string filename)
137 {
138     Graph G = ReadGraph(filename);
139     Matching M(G);
140
141     list<int> matching;
142     matching = M.SolveMaximumMatching();
143
144     cout << "Number of edges in the maximum matching: " <<
145 matching.size() << endl;
146     cout << "Edges in the matching:" << endl;
147     for(list<int>::iterator it = matching.begin(); it !=
148 matching.end(); it++)
149     {
150         pair<int, int> e = G.GetEdge( *it );
151
152         cout << e.first << " " << e.second << endl;
153     }
154 }
155
156 int main(int argc, char* argv[])
157 {
158     string filename = "";
159     string algorithm = "";
160
161     int i = 1;
162     while(i < argc)

```

```

    {
        string a(argv[i]);
        if(a == "-f")
            filename = argv[++i];
        else if(a == "--minweight")
            algorithm = "minweight";
        else if(a == "--max")
            algorithm = "max";
163
164        i++;
165    }
166
167    if(filename == "" || algorithm == "")
168    {
169        cout << "usage: ./example -f <filename> <--minweight |
170 --max>" << endl;
171        cout << "--minweight for minimum weight perfect
172 matching" << endl;
173        cout << "--max for maximum cardinality matching" <<
174 endl;
175        cout << "file format:" << endl;
176        cout <<
177 "the first two lines give n (number of vertices) and m (number of
178 edges)," << endl;
179        cout <<
180 "followed by m lines, each with a tuple (u, v [, c]) representing the
181 edges," << endl;
182        cout << "where u and v are the endpoints (0-based
183 indexing) of the edge and c is its cost" << endl;
184        cout << "the cost is optional if --max is specified" <<
185 endl;
186        return 1;
187    }
188
189    try
190    {
191        if(algorithm == "minweight")
192            MinimumCostPerfectMatchingExample(filename);
193        else
194            MaximumMatchingExample(filename);
195    }
196    catch(const char * msg)
197    {
        cout << msg << endl;
        return 1;
    }

    return 0;
}

```

Globals.h

```

1 #pragma once
2
3 #define EPSILON 0.000001
4 #define INFINITO 1000000000.0
5 #define GREATER(A, B) ((A) - (B) > EPSILON)
6 #define LESS(A, B) ((B) - (A) > EPSILON)
7 #define EQUAL(A, B) (fabs((A) - (B)) < EPSILON)
8 #define GREATER_EQUAL(A, B) (GREATER((A),(B)) || EQUAL((A),(B)))
9 #define LESS_EQUAL(A, B) (LESS((A),(B)) || EQUAL((A),(B)))

```

```

10 #define MIN(A, B) (LESS((A),(B)) ? (A) : (B))
11 #define MAX(A, B) (LESS((A),(B)) ? (B) : (A))

```

Graph.cpp

```

1  #include "Graph.h"
2
3  Graph::Graph(int n, const list< pair<int, int> > & edges):
4      n(n),
5      m(edges.size()),
6      adjMat(n, vector<bool>(n, false)),
7      adjList(n),
8      edges(),
9      edgeIndex(n, vector<int>(n, -1))
10 {
11     for(list< pair<int, int> >::const_iterator it = edges.begin();
12 it != edges.end(); it++)
13     {
14         int u = (*it).first;
15         int v = (*it).second;
16
17         AddEdge(u, v);
18     }
19 }
20
21 pair<int, int> Graph::GetEdge(int e) const
22 {
23     if(e > (int)edges.size())
24         throw "Error: edge does not exist";
25
26     return edges[e];
27 }
28
29 int Graph::GetEdgeIndex(int u, int v) const
30 {
31     if( u > n or
32        v > n )
33         throw "Error: vertex does not exist";
34
35     if(edgeIndex[u][v] == -1)
36         throw "Error: edge does not exist";
37
38     return edgeIndex[u][v];
39 }
40
41 void Graph::AddVertex()
42 {
43     for(int i = 0; i < n; i++)
44     {
45         adjMat[i].push_back(false);
46         edgeIndex[i].push_back(-1);
47     }
48     n++;
49     adjMat.push_back( vector<bool>(n, false) );
50     edgeIndex.push_back( vector<int>(n, -1) );
51     adjList.push_back( list<int>() );
52 }
53
54 void Graph::AddEdge(int u, int v)
55 {

```

```

56         if( u > n or
57             v > n )
58             throw "Error: vertex does not exist";
59
60         if(adjMat[u][v]) return;
61
62         adjMat[u][v] = adjMat[v][u] = true;
63         adjList[u].push_back(v);
64         adjList[v].push_back(u);
65
66         edges.push_back(pair<int, int>(u, v));
67         edgeIndex[u][v] = edgeIndex[v][u] = m++;
68     }
69
70     const list<int> & Graph::AdjList(int v) const
71     {
72         if(v > n)
73             throw "Error: vertex does not exist";
74
75         return adjList[v];
76     }
77
78     const vector< vector<bool> > & Graph::AdjMat() const
79     {
80         return adjMat;
81     }

```

Graph.h

```

1  #pragma once
2
3  #include <list>
4  #include <vector>
5  using namespace std;
6
7  class Graph
8  {
9  public:
10     //n is the number of vertices
11     //edges is a list of pairs representing the edges (default =
12     empty list)
13     Graph(int n, const list< pair<int, int> > & edges = list<
14     pair<int, int> >());
15
16     //Default constructor creates an empty graph
17     Graph(): n(0), m(0) {};
18
19     //Returns the number of vertices
20     int GetNumVertices() const { return n; };
21     //Returns the number of edges
22     int GetNumEdges() const { return m; };
23
24     //Given the edge's index, returns its endpoints as a pair
25     pair<int, int> GetEdge(int e) const;
26     //Given the endpoints, returns the index
27     int GetEdgeIndex(int u, int v) const;
28
29     //Adds a new vertex to the graph
30     void AddVertex();
31     //Adds a new edge to the graph

```

```

32         void AddEdge(int u, int v);
33         //Returns the adjacency list of a vertex
34         const list<int> & AdjList(int v) const;
35
36         //Returns the graph's adjacency matrix
37         const vector< vector<bool> > & AdjMat() const;
38     private:
39         //Number of vertices
40         int n;
41         //Number of edges
42         int m;
43
44         //Adjacency matrix
45         vector< vector<bool> > adjMat;
46
47         //Adjacency lists
48         vector< list<int> > adjList;
49
50         //Array of edges
51         vector< pair<int, int> > edges;
52
53         //Indices of the edges
54         vector< vector<int> > edgeIndex;
55     };

```

Matching.cpp

```

1  #include "Matching.h"
2
3  Matching::Matching(const Graph & G):
4      G(G),
5      outer(2*G.GetNumVertices()),
6      deep(2*G.GetNumVertices()),
7      shallow(2*G.GetNumVertices()),
8      tip(2*G.GetNumVertices()),
9      active(2*G.GetNumVertices()),
10     type(2*G.GetNumVertices()),
11     forest(2*G.GetNumVertices()),
12     root(2*G.GetNumVertices()),
13     blocked(2*G.GetNumVertices()),
14     dual(2*G.GetNumVertices()),
15     slack(G.GetNumEdges()),
16     mate(2*G.GetNumVertices()),
17     m(G.GetNumEdges()),
18     n(G.GetNumVertices()),
19     visited(2*G.GetNumVertices())
20 {
21 }
22
23 void Matching::Grow()
24 {
25     Reset();
26
27     //All unmatched vertices will be roots in a forest that will be
28     grown
29     //The forest is grown by extending a unmatched vertex w through
30     a matched edge u-v in a BFS fashion
31     while(!forestList.empty())
32     {

```

```

33         int w = outer[forestList.front()];
34         forestList.pop_front();
35
36         //w might be a blossom
37         //we have to explore all the connections from vertices
38         inside the blossom to other vertices
39         for(list<int>::iterator it = deep[w].begin(); it !=
40 deep[w].end(); it++)
41         {
42             int u = *it;
43
44             int cont = false;
45             for(list<int>::const_iterator jt =
46 G.AdjList(u).begin(); jt != G.AdjList(u).end(); jt++)
47             {
48                 int v = *jt;
49
50                 if(IsEdgeBlocked(u, v)) continue;
51
52                 //u is even and v is odd
53                 if(type[outer[v]] == ODD)
54                     continue;
55
56                 //if v is unlabeled
57                 if(type[outer[v]] != EVEN)
58                 {
59                     //We grow the alternating forest
60                     int vm = mate[outer[v]];
61
62                     forest[outer[v]] = u;
63                     type[outer[v]] = ODD;
64                     root[outer[v]] = root[outer[u]];
65                     forest[outer[vm]] = v;
66                     type[outer[vm]] = EVEN;
67                     root[outer[vm]] =
68 root[outer[u]];
69
70                     if(!visited[outer[vm]])
71                     {
72
73 forestList.push_back(vm);
74
75 true;
76
77                     }
78                     //If v is even and u and v are on
79 different trees
80                     //we found an augmenting path
81                     else if(root[outer[v]] !=
82 root[outer[u]])
83                     {
84                         Augment(u,v);
85                         Reset();
86
87                         cont = true;
88                         break;
89                     }
90                     //If u and v are even and on the same
91 tree
92                     //we found a blossom
93                     else if(outer[u] != outer[v])
94                     {
95                         int b = Blossom(u,v);
96

```

```

97         forestList.push_front(b);
98         visited[b] = true;
99
100         cont = true;
101         break;
102     }
103 }
104     if(cont) break;
105 }
106 }
107
108 //Check whether the matching is perfect
109 perfect = true;
110 for(int i = 0; i < n; i++)
111     if(mate[outer[i]] == -1)
112         perfect = false;
113 }
114
115 bool Matching::IsAdjacent(int u, int v)
116 {
117     return (G.AdjMat()[u][v] and not IsEdgeBlocked(u, v));
118 }
119
120 bool Matching::IsEdgeBlocked(int u, int v)
121 {
122     return GREATER(slack[ G.GetEdgeIndex(u, v) ], 0);
123 }
124
125 bool Matching::IsEdgeBlocked(int e)
126 {
127     return GREATER(slack[e], 0);
128 }
129
130 //Vertices will be selected in non-decreasing order of their degree
131 //Each time an unmatched vertex is selected, it is matched to its
132 adjacent unmatched vertex of minimum degree
133 void Matching::Heuristic()
134 {
135     vector<int> degree(n, 0);
136     BinaryHeap B;
137
138     for(int i = 0; i < m; i++)
139     {
140         if(IsEdgeBlocked(i)) continue;
141
142         pair<int, int> p = G.GetEdge(i);
143         int u = p.first;
144         int v = p.second;
145
146         degree[u]++;
147         degree[v]++;
148     }
149
150     for(int i = 0; i < n; i++)
151         B.Insert(degree[i], i);
152
153     while(B.Size() > 0)
154     {
155         int u = B.DeleteMin();
156         if(mate[outer[u]] == -1)
157         {
158             int min = -1;
159             for(list<int>::const_iterator it =
160 G.AdjList(u).begin(); it != G.AdjList(u).end(); it++)

```

```

161         {
162             int v = *it;
163
164             if(IsEdgeBlocked(u, v) or
165                (outer[u] == outer[v]) or
166                (mate[outer[v]] != -1) )
167                 continue;
168
169             if(min == -1 or degree[v] < degree[min])
170                 min = v;
171         }
172         if(min != -1)
173         {
174             mate[outer[u]] = min;
175             mate[outer[min]] = u;
176         }
177     }
178 }
179 }
180
181 //Destroys a blossom recursively
182 void Matching::DestroyBlossom(int t)
183 {
184     if((t < n) or
185        (blocked[t] and GREATER(dual[t], 0))) return;
186
187     for(list<int>::iterator it = shallow[t].begin(); it !=
188 shallow[t].end(); it++)
189     {
190         int s = *it;
191         outer[s] = s;
192         for(list<int>::iterator jt = deep[s].begin(); jt !=
193 deep[s].end(); jt++)
194             outer[*jt] = s;
195
196         DestroyBlossom(s);
197     }
198
199     active[t] = false;
200     blocked[t] = false;
201     AddFreeBlossomIndex(t);
202     mate[t] = -1;
203 }
204
205 void Matching::Expand(int u, bool expandBlocked = false)
206 {
207     int v = outer[mate[u]];
208
209     int index = m;
210     int p = -1, q = -1;
211     //Find the regular edge {p,q} of minimum index connecting u and
212     its mate
213     //We use the minimum index to grant that the two possible
214     blossoms u and v will use the same edge for a mate
215     for(list<int>::iterator it = deep[u].begin(); it !=
216 deep[u].end(); it++)
217     {
218         int di = *it;
219         for(list<int>::iterator jt = deep[v].begin(); jt !=
220 deep[v].end(); jt++)
221         {
222             int dj = *jt;
223             if(IsAdjacent(di, dj) and G.GetEdgeIndex(di,
224 dj) < index)

```

```

225         {
226             index = G.GetEdgeIndex(di, dj);
227             p = di;
228             q = dj;
229         }
230     }
231 }
232
233     mate[u] = q;
234     mate[v] = p;
235     //If u is a regular vertex, we are done
236     if(u < n or (blocked[u] and not expandBlocked)) return;
237
238     bool found = false;
239     //Find the position t of the new tip of the blossom
240     for(list<int>::iterator it = shallow[u].begin(); it !=
241 shallow[u].end() and not found; )
242     {
243         int si = *it;
244         for(list<int>::iterator jt = deep[si].begin(); jt !=
245 deep[si].end() and not found; jt++)
246         {
247             if(*jt == p )
248                 found = true;
249         }
250         it++;
251         if(not found)
252         {
253             shallow[u].push_back(si);
254             shallow[u].pop_front();
255         }
256     }
257
258     list<int>::iterator it = shallow[u].begin();
259     //Adjust the mate of the tip
260     mate[*it] = mate[u];
261     it++;
262     //
263     //Now we go through the odd circuit adjusting the new mates
264     while(it != shallow[u].end())
265     {
266         list<int>::iterator itnext = it;
267         itnext++;
268         mate[*it] = *itnext;
269         mate[*itnext] = *it;
270         itnext++;
271         it = itnext;
272     }
273
274     //We update the sets blossom, shallow, and outer since this
275 blossom is being deactivated
276     for(list<int>::iterator it = shallow[u].begin(); it !=
277 shallow[u].end(); it++)
278     {
279         int s = *it;
280         outer[s] = s;
281         for(list<int>::iterator jt = deep[s].begin(); jt !=
282 deep[s].end(); jt++)
283             outer[*jt] = s;
284     }
285     active[u] = false;
286     AddFreeBlossomIndex(u);
287
288     //Expand the vertices in the blossom

```

```

289         for(list<int>::iterator it = shallow[u].begin(); it !=
290 shallow[u].end(); it++)
291             Expand(*it, expandBlocked);
292     }
293 }
294
295 //Augment the path root[u], ..., u, v, ..., root[v]
296 void Matching::Augment(int u, int v)
297 {
298     //We go from u and v to its respective roots, alternating the
299 matching
300     int p = outer[u];
301     int q = outer[v];
302     int outv = q;
303     int fp = forest[p];
304     mate[p] = q;
305     mate[q] = p;
306     Expand(p);
307     Expand(q);
308     while(fp != -1)
309     {
310         q = outer[forest[p]];
311         p = outer[forest[q]];
312         fp = forest[p];
313
314         mate[p] = q;
315         mate[q] = p;
316         Expand(p);
317         Expand(q);
318     }
319
320     p = outv;
321     fp = forest[p];
322     while(fp != -1)
323     {
324         q = outer[forest[p]];
325         p = outer[forest[q]];
326         fp = forest[p];
327
328         mate[p] = q;
329         mate[q] = p;
330         Expand(p);
331         Expand(q);
332     }
333 }
334
335 void Matching::Reset()
336 {
337     for(int i = 0; i < 2*n; i++)
338     {
339         forest[i] = -1;
340         root[i] = i;
341
342         if(i >= n and active[i] and outer[i] == i)
343             DestroyBlossom(i);
344     }
345
346     visited.assign(2*n, 0);
347     forestList.clear();
348     for(int i = 0; i < n; i++)
349     {
350         if(mate[outer[i]] == -1)
351         {
352             type[outer[i]] = 2;

```

```

353             if(!visited[outer[i]])
354                 forestList.push_back(i);
355             visited[outer[i]] = true;
356         }
357         else type[outer[i]] = 0;
358     }
359 }
360
361 int Matching::GetFreeBlossomIndex()
362 {
363     int i = free.back();
364     free.pop_back();
365     return i;
366 }
367
368 void Matching::AddFreeBlossomIndex(int i)
369 {
370     free.push_back(i);
371 }
372
373 void Matching::ClearBlossomIndices()
374 {
375     free.clear();
376     for(int i = n; i < 2*n; i++)
377         AddFreeBlossomIndex(i);
378 }
379
380 //Contracts the blossom w, ..., u, v, ..., w, where w is the first
381 vertex that appears in the paths from u and v to their respective roots
382 int Matching::Blossom(int u, int v)
383 {
384     int t = GetFreeBlossomIndex();
385
386     vector<bool> isInPath(2*n, false);
387
388     //Find the tip of the blossom
389     int u_ = u;
390     while(u_ != -1)
391     {
392         isInPath[outer[u_]] = true;
393
394         u_ = forest[outer[u_]];
395     }
396
397     int v_ = outer[v];
398     while(not isInPath[v_])
399         v_ = outer[forest[v_]];
400     tip[t] = v_;
401
402     //Find the odd circuit, update shallow, outer, blossom and deep
403     //First we construct the set shallow (the odd circuit)
404     list<int> circuit;
405     u_ = outer[u];
406     circuit.push_front(u_);
407     while(u_ != tip[t])
408     {
409         u_ = outer[forest[u_]];
410         circuit.push_front(u_);
411     }
412
413     shallow[t].clear();
414     deep[t].clear();
415     for(list<int>::iterator it = circuit.begin(); it !=
416 circuit.end(); it++)

```

```

417         {
418             shallow[t].push_back(*it);
419         }
420
421         v_ = outer[v];
422         while(v_ != tip[t])
423         {
424             shallow[t].push_back(v_);
425             v_ = outer[forest[v_]];
426         }
427
428         //Now we construct deep and update outer
429         for(list<int>::iterator it = shallow[t].begin(); it !=
430 shallow[t].end(); it++)
431         {
432             u_ = *it;
433             outer[u_] = t;
434             for(list<int>::iterator jt = deep[u_].begin(); jt !=
435 deep[u_].end(); jt++)
436             {
437                 deep[t].push_back(*jt);
438                 outer[*jt] = t;
439             }
440         }
441
442         forest[t] = forest[tip[t]];
443         type[t] = EVEN;
444         root[t] = root[tip[t]];
445         active[t] = true;
446         outer[t] = t;
447         mate[t] = mate[tip[t]];
448
449         return t;
450     }
451
452     void Matching::UpdateDualCosts()
453     {
454         double e1 = 0, e2 = 0, e3 = 0;
455         int inite1 = false, inite2 = false, inite3 = false;
456         for(int i = 0; i < m; i++)
457         {
458             int u = G.GetEdge(i).first,
459                 v = G.GetEdge(i).second;
460
461             if( (type[outer[u]] == EVEN and type[outer[v]] ==
462 UNLABELED) or (type[outer[v]] == EVEN and type[outer[u]] == UNLABELED) )
463             {
464                 if(!inite1 or GREATER(e1, slack[i]))
465                 {
466                     e1 = slack[i];
467                     inite1 = true;
468                 }
469             }
470             else if( (outer[u] != outer[v]) and type[outer[u]] ==
471 EVEN and type[outer[v]] == EVEN )
472             {
473                 if(!inite2 or GREATER(e2, slack[i]))
474                 {
475                     e2 = slack[i];
476                     inite2 = true;
477                 }
478             }
479         }
480         for(int i = n; i < 2*n; i++)

```

```

481     {
482         if(active[i] and i == outer[i] and type[outer[i]] ==
483 ODD and (!inite3 or GREATER(e3, dual[i])))
484         {
485             e3 = dual[i];
486             inite3 = true;
487         }
488     }
489     double e = 0;
490     if(inite1) e = e1;
491     else if(inite2) e = e2;
492     else if(inite3) e = e3;
493
494     if(GREATER(e, e2/2.0) and inite2)
495         e = e2/2.0;
496     if(GREATER(e, e3) and inite3)
497         e = e3;
498
499     for(int i = 0; i < 2*n; i++)
500     {
501         if(i != outer[i]) continue;
502
503         if(active[i] and type[outer[i]] == EVEN)
504         {
505             dual[i] += e;
506         }
507         else if(active[i] and type[outer[i]] == ODD)
508         {
509             dual[i] -= e;
510         }
511     }
512
513     for(int i = 0; i < m; i++)
514     {
515         int u = G.GetEdge(i).first,
516             v = G.GetEdge(i).second;
517
518         if(outer[u] != outer[v])
519         {
520             if(type[outer[u]] == EVEN and type[outer[v]] ==
521 EVEN)
522                 slack[i] -= 2.0*e;
523             else if(type[outer[u]] == ODD and
524 type[outer[v]] == ODD)
525                 slack[i] += 2.0*e;
526             else if( (type[outer[v]] == UNLABELED and
527 type[outer[u]] == EVEN) or (type[outer[u]] == UNLABELED and
528 type[outer[v]] == EVEN) )
529                 slack[i] -= e;
530             else if( (type[outer[v]] == UNLABELED and
531 type[outer[u]] == ODD) or (type[outer[u]] == UNLABELED and
532 type[outer[v]] == ODD) )
533                 slack[i] += e;
534         }
535     }
536     for(int i = n; i < 2*n; i++)
537     {
538         if(GREATER(dual[i], 0))
539         {
540             blocked[i] = true;
541         }
542         else if(active[i] and blocked[i])
543         {
544             //The blossom is becoming unblocked

```

```

545             if(mate[i] == -1)
546             {
547                 DestroyBlossom(i);
548             }
549             else
550             {
551                 blocked[i] = false;
552                 Expand(i);
553             }
554         }
555     }
556 }
557
558 pair< list<int>, double>
559 Matching::SolveMinimumCostPerfectMatching(const vector<double> & cost)
560 {
561     SolveMaximumMatching();
562     if(!perfect)
563         throw "Error: The graph does not have a perfect
564 matching";
565
566     Clear();
567
568     //Initialize slacks (reduced costs for the edges)
569     slack = cost;
570
571     PositiveCosts();
572
573     //If the matching on the compressed graph is perfect, we are
574 done
575     perfect = false;
576     while(not perfect)
577     {
578         //Run an heuristic maximum matching algorithm
579         Heuristic();
580         //Grow a hungarian forest
581         Grow();
582         UpdateDualCosts();
583         //Set up the algorithm for a new grow step
584         Reset();
585     }
586
587     list<int> matching = RetrieveMatching();
588
589     double obj = 0;
590     for(list<int>::iterator it = matching.begin(); it !=
591 matching.end(); it++)
592         obj += cost[*it];
593
594     double dualObj = 0;
595     for(int i = 0; i < 2*n; i++)
596     {
597         if(i < n) dualObj += dual[i];
598         else if(blocked[i]) dualObj += dual[i];
599     }
600
601     return pair< list<int>, double >(matching, obj);
602 }
603
604 void Matching::PositiveCosts()
605 {
606     double minEdge = 0;
607     for(int i = 0; i < m ;i++)
608         if(GREATER(minEdge - slack[i], 0))

```

```

        minEdge = slack[i];

        for(int i = 0; i < m; i++)
            slack[i] -= minEdge;
    }

    list<int> Matching::SolveMaximumMatching()
    {
        Clear();
        Grow();
        return RetrieveMatching();
    }

    //Sets up the algorithm for a new run
    void Matching::Clear()
    {
        ClearBlossomIndices();

        for(int i = 0; i < 2*n; i++)
        {
            outer[i] = i;
            deep[i].clear();
            if(i<n)
                deep[i].push_back(i);
            shallow[i].clear();
            if(i < n) active[i] = true;
            else active[i] = false;

            type[i] = 0;
            forest[i] = -1;
            root[i] = i;

            blocked[i] = false;
            dual[i] = 0;
            mate[i] = -1;
            tip[i] = i;
        }
        slack.assign(m, 0);
    }

    list<int> Matching::RetrieveMatching()
    {
        list<int> matching;

        for(int i = 0; i < 2*n; i++)
            if(active[i] and mate[i] != -1 and outer[i] == i)
                Expand(i, true);

        for(int i = 0; i < m; i++)
        {
            int u = G.GetEdge(i).first;
            int v = G.GetEdge(i).second;

            if(mate[u] == v)
                matching.push_back(i);
        }
        return matching;
    }

```

```

1  #pragma once
2
3  #include "Graph.h"
4  #include "BinaryHeap.h"
5  #include <list>
6  #include <vector>
7  using namespace std;
8
9  #define EVEN 2
10 #define ODD 1
11 #define UNLABELED 0
12
13 class Matching
14 {
15 public:
16     //Parametric constructor receives a graph instance
17     Matching(const Graph & G);
18
19     //Solves the minimum cost perfect matching problem
20     //Receives the a vector whose position i has the cost of the edge
21 with index i
22     //If the graph does not have a perfect matching, a const char *
23 exception will be raised
24     //Returns a pair
25     //the first element of the pair is a list of the indices of the
26 edges in the matching
27     //the second is the cost of the matching
28     pair< list<int>, double > SolveMinimumCostPerfectMatching(const
29 vector<double> & cost);
30
31     //Solves the maximum cardinality matching problem
32     //Returns a list with the indices of the edges in the matching
33     list<int> SolveMaximumMatching();
34
35 private:
36     //Grows an alternating forest
37     void Grow();
38     //Expands a blossom u
39     //If expandBlocked is true, the blossom will be expanded even if
40 it is blocked
41     void Expand(int u, bool expandBlocked);
42     //Augments the matching using the path from u to v in the
43 alternating forest
44     void Augment(int u, int v);
45     //Resets the alternating forest
46     void Reset();
47     //Creates a blossom where the tip is the first common vertex in
48 the paths from u and v in the hungarian forest
49     int Blossom(int u, int v);
50     void UpdateDualCosts();
51     //Resets all data structures
52     void Clear();
53     void DestroyBlossom(int t);
54     //Uses an heuristic algorithm to find the maximum matching of the
55 graph
56     void Heuristic();
57     //Modifies the costs of the graph so the all edges have positive
58 costs
59     void PositiveCosts();
60     list<int> RetrieveMatching();
61
62     int GetFreeBlossomIndex();
63     void AddFreeBlossomIndex(int i);

```

```

void ClearBlossomIndices();

//An edge might be blocked due to the dual costs
bool IsEdgeBlocked(int u, int v);
bool IsEdgeBlocked(int e);
//Returns true if u and v are adjacent in G and not blocked
bool IsAdjacent(int u, int v);

const Graph & G;

list<int> free;//List of free blossom indices
64
65
66 vector<int> outer;//outer[v] gives the index of the outermost
67 blossom that contains v, outer[v] = v if v is not contained in any
68 blossom
69 vector< list<int> > deep;//deep[v] is a list of all the original
70 vertices contained inside v, deep[v] = v if v is an original vertex
71 vector< list<int> > shallow;//shallow[v] is a list of the
72 vertices immediately contained inside v, shallow[v] is empty is the
73 default
74 vector<int> tip;//tip[v] is the tip of blossom v
75 vector<bool> active;//true if a blossom is being used
76
77 vector<int> type;//Even, odd, neither (2, 1, 0)
78 vector<int> forest;//forest[v] gives the father of v in the
79 alternating forest
80 vector<int>
81 root;//root[v] gives the root of v in the alternating forest
82
83 vector<bool> blocked;//A blossom can be blocked due to dual
84 costs, this means that it behaves as if it were an original vertex and
85 cannot be expanded
86 vector<double> dual;//dual multipliers associated to the
87 blossoms, if dual[v] > 0, the blossom is blocked and full
88 vector<double>
89 slack;//slack associated to each edge, if slack[e] > 0, the edge cannot
90 be used
91 vector<int> mate;//mate[v] gives the mate of v
92
93 int m, n;
94
95 bool perfect;
96
97 list<int> forestList;
98 vector<int> visited;
99
100 };

```

Repository Code Listing

ChinesePostman.h

```
1  #include "../Minimum-Cost-Perfect-Matching/Graph.h"
2  #include "../Minimum-Cost-Perfect-Matching/Matching.h"
3  #include "../Dijkstra.h"
4
5  bool Connected(const Graph &G) {
6      vector<bool> visited(G.GetNumVertices(), false);
7      list<int> L;
8
9      int n = 0;
10     L.push_back(0);
11     while (not L.empty()) {
12         int u = L.back();
13         L.pop_back();
14         if (visited[u])
15             continue;
16
17         visited[u] = true;
18         n++;
19
20         for (list<int>::const_iterator it = G.AdjList(u).begin();
21              it != G.AdjList(u).end(); it++) {
22             int v = *it;
23             L.push_back(v);
24         }
25     }
26
27     return G.GetNumVertices() == n;
28 }
29
30 /*
31  Solves the chinese postman problem
32  returns a pair containing a list and a double
33  the list is the sequence of vertices in the solution
34  the double is the solution cost
35  */
36 pair<list<int>, double> ChinesePostman(const Graph &G,
37                                         const vector<double> &cost) {
38     // Check if the graph is connected
39     if (not Connected(G))
40         throw "Error: Graph is not connected";
41
42     // Build adjacency lists using edges in the graph
43     vector<list<int>> A(G.GetNumVertices(), list<int>());
44     for (int u = 0; u < G.GetNumVertices(); u++)
45         A[u] = G.AdjList(u);
46
47     // Find vertices with odd degree
48     vector<int> odd;
49     for (int u = 0; u < G.GetNumVertices(); u++)
50         if (A[u].size() % 2)
51             odd.push_back(u);
52
53     // If there are odd degree vertices
54     if (not odd.empty()) {
```

```

55 // Create a graph with the odd degree vertices
56 Graph O(odd.size());
57 for (int u = 0; u < (int)odd.size(); u++)
58     for (int v = u + 1; v < (int)odd.size(); v++)
59         O.AddEdge(u, v);
60
61 vector<double> costO(O.GetNumEdges());
62
63 // Find the shortest paths between all odd degree vertices
64 vector<vector<int>> shortestPath(O.GetNumVertices());
65 for (int u = 0; u < (int)odd.size(); u++) {
66     pair<vector<int>, vector<double>> sp = Dijkstra(G, odd[u], cost);
67
68     shortestPath[u] = sp.first;
69
70     // The cost of an edge uv in O will be the cost of the
71 corresponding
72 // shortest path in G
73     for (int v = 0; v < (int)odd.size(); v++)
74         if (v != u)
75             costO[O.GetEdgeIndex(u, v)] = sp.second[odd[v]];
76 }
77
78 // Find the minimum cost perfect matching of the graph of the odd
79 degree
80 // vertices
81 Matching M(O);
82 pair<list<int>, double> p =
83 M.SolveMinimumCostPerfectMatching(costO);
84 list<int> matching = p.first;
85
86 // If an edge uv is in the matching, the edges in the shortest path
87 from u
88 // to v should be duplicated in G
89 for (list<int>::iterator it = matching.begin(); it !=
90 matching.end();
91     it++) {
92     pair<int, int> p = O.GetEdge(*it);
93     int u = p.first, v = odd[p.second];
94
95     // Go through the path adding the edges
96     int w = shortestPath[u][v];
97     while (w != -1) {
98         A[w].push_back(v);
99         A[v].push_back(w);
100         v = w;
101         w = shortestPath[u][v];
102     }
103 }
104 }
105
106 // Find an Eulerian cycle in the graph implied by A
107 list<int> cycle;
108 // This is to keep track of how many times we can traverse an edge
109 vector<int> traversed(G.GetNumEdges(), 0);
110 for (int u = 0; u < G.GetNumVertices(); u++) {
111     for (list<int>::iterator it = A[u].begin(); it != A[u].end(); it++)
112     {
113         int v = *it;
114
115         // we do this so that the edge is not counted twice
116         if (v < u)
117             continue;
118

```

```

        traversed[G.GetEdgeIndex(u, v)]++;
    }
}

119 cycle.push_back(0);
120 list<int>::iterator itp = cycle.begin();
121 double obj = 0;
122 while (itp != cycle.end()) {
123     // Let u be the current vertex in the cycle, starting at the first
124     int u = *itp;
125     list<int>::iterator jtp = itp;
126     jtp++;
127
128     // if there are non-traversed edges incident to u, find a subcycle
129     starting
130     // at u replace u in the cycle by the subcycle
131     while (not A[u].empty()) {
132         while (not A[u].empty() and
133             traversed[G.GetEdgeIndex(u, A[u].back())] == 0)
134             A[u].pop_back();
135
136         if (not A[u].empty()) {
137             int v = A[u].back();
138             A[u].pop_back();
139             cycle.insert(jtp, v);
140             traversed[G.GetEdgeIndex(u, v)]--;
141             obj += cost[G.GetEdgeIndex(u, v)];
142             u = v;
143         }
144     }
145
146     // go to the next vertex in the cycle and do the same
147     itp++;
148 }

return pair<list<int>, double>(cycle, obj);
}

```

Dijkstra.h

```

1 #pragma once
2
3 #include "../Minimum-Cost-Perfect-Matching/BinaryHeap.h"
4 #include "../Minimum-Cost-Perfect-Matching/Globals.h"
5 #include "../Minimum-Cost-Perfect-Matching/Graph.h"
6 #include <limits>
7 using namespace std;
8
9 // Dijkstra's algorithm using binary heap
10 // Returns a pair (vector<int>, vector<double>)
11 // vector<double> gives the cost of the optimal path to each vertex
12 // vector<int> gives the parent of each vertex in the tree of optimal
13 paths
14 pair<vector<int>, vector<double>> Dijkstra(const Graph &G, int origin,
15                                           const vector<double> &cost) {
16     BinaryHeap B;
17
18     int n = G.GetNumVertices();
19
20     // Father of each vertex in the optimal path tree

```

```

21     vector<int> father(n, -1);
22     // Used to indicate whether a vertex is permanently labeled
23     vector<bool> permanent(n, false);
24
25     vector<double> pathCost(n, numeric_limits<double>::infinity());
26
27     // Put s in the heap
28     B.Insert(0, origin);
29     pathCost[origin] = 0;
30
31     for (int i = 0; i < n; i++) {
32         // Select the vertex that can be reached with smallest cost
33         int u = B.DeleteMin();
34
35         permanent[u] = true;
36
37         // Update the heap with vertices adjacent to u
38         for (list<int>::const_iterator it = G.AdjList(u).begin();
39              it != G.AdjList(u).end(); it++) {
40             int v = *it;
41
42             if (permanent[v])
43                 continue;
44
45             double c = pathCost[u] + cost[G.GetEdgeIndex(u, v)];
46
47             // v has not been discovered yet
48             if (father[v] == -1) {
49                 father[v] = u;
50                 pathCost[v] = c;
51                 B.Insert(c, v);
52             }
53             // we found a cheaper connection to v
54             else if (LESS(c, pathCost[v])) {
55                 father[v] = u;
56                 pathCost[v] = c;
57                 B.ChangeKey(c, v);
58             }
59         }
60     }
61
62     if (B.Size() > 0)
63         throw "Error: graph is not connected";
64
65     return make_pair(father, pathCost);
66 }

```

Example.cpp

```

1  #include "../Minimum-Cost-Perfect-Matching/Graph.h"
2  #include "ChinesePostman.h"
3  #include <fstream>
4  #include <iostream>
5  #include <sstream>
6  #include <string>
7  using namespace std;
8
9  pair<Graph, vector<double>> ReadWeightedGraph(string filename) {
10     // Please see Graph.h for a description of the interface

```

```

11
12 ifstream file;
13 file.open(filename.c_str());
14
15 string s;
16 getline(file, s);
17 stringstream ss(s);
18 int n;
19 ss >> n;
20 getline(file, s);
21 ss.str(s);
22 ss.clear();
23 int m;
24 ss >> m;
25
26 Graph G(n);
27 vector<double> cost(m);
28 for (int i = 0; i < m; i++) {
29     getline(file, s);
30     ss.str(s);
31     ss.clear();
32     int u, v;
33     double c;
34     ss >> u >> v >> c;
35
36     G.AddEdge(u, v);
37     cost[G.GetEdgeIndex(u, v)] = c;
38 }
39
40 file.close();
41 return make_pair(G, cost);
42 }
43
44 int main(int argc, char *argv[]) {
45     string filename = "";
46
47     int i = 1;
48     while (i < argc) {
49         string a(argv[i]);
50         if (a == "-f")
51             filename = argv[++i];
52         i++;
53     }
54
55     if (filename == "") {
56         cout << endl << "usage: ./example -f <filename>" << endl << endl;
57         cout << "-f followed by file name to specify the input file." <<
58 endl
59         << endl;
60
61         cout << "File format:" << endl;
62         cout << "The first two lines give n (number of vertices) and m
63 (number of "
64         << "edges),"
65         << endl;
66         cout << "Each of the next m lines has a tuple (u, v [, c])
67 representing an "
68         << "edge,"
69         << endl;
70         cout << "where u and v are the endpoints (0-based indexing) of the
71 edge "
72         << "and c is its cost"
73         << endl;
74     return 1;

```

```

    }

75     try {
76         Graph G;
77         vector<double> cost;
78
79         // Read the graph
80         pair<Graph, vector<double>> p = ReadWeightedGraph(filename);
81         G = p.first;
82         cost = p.second;
83
84         // Solve the problem
85         pair<list<int>, double> sol = ChinesePostman(G, cost);
86
87         cout << "Solution cost: " << sol.second << endl;
88
89         list<int> s = sol.first;
90
91         // Print edges in the solution
92         cout << "Solution:" << endl;
93         for (list<int>::iterator it = s.begin(); it != s.end(); it++)
94             cout << *it << " ";
95         cout << endl;
96     } catch (const char *msg) {
97         cout << msg << endl;
98         return 1;
99     }
100
    return 0;
}

```